

Sleep-Time Compute

새로운 학습·기억 **scaling** 축의 조건부 평가

근거 maturity, strongest alternatives, scaling hypotheses,
system infrastructure 와 memory-device opportunity

핵심 결론. External-memory background synthesis 는 이미 production lifecycle 로 나타났지만, 광범위한 per-user foundation-weight sleep 은 아직 조건부 연구 가설이다. 가장 신뢰할 중기 구조는 auditable external system-of-record 와 selective, reversible parametric promotion 을 결합하는 hybrid 다. 본 논문의 scaling 식은 보편 법칙이 아니라 accounting identity, break-even model, fit candidate, falsifiable hypothesis 로 등급을 분리한다.

독자와 범위

이 문서는 training 을 전문으로 하지 않는 system-memory 연구자가 논문의 주장과 workload 를 끝까지 검증할 수 있도록 작성했다. 첫 등장 training 개념의 [배경: concept-id] 링크는 별도 57 쪽 *Training Background* PDF 의 정확한 정의로 연결된다. 본문 claim ID 는 마지막 canonical claim-evidence ledger 로, source citation 은 2026-08-05 에 동결한 73 개 primary/official source 로 연결된다.

핵심 이 문서가 주장하지 않는 것

“더 많은 sleep FLOPs 가 항상 더 좋은 기억을 만든다” 는 universal scaling law, biological sleep 의 직접 복제 필요성, weights 가 external memory 보다 본질적으로 우월하다는 결론, 특정 device market size 는 주장하지 않는다. 현재 evidence 와 original hypothesis 를 시각·표기상 분리한다.

AI assistance disclosure. During the preparation of this work the authors used Codex GPT-5.6 for evidence synthesis, LaTeX drafting, and artifact QA. All content was reviewed and edited by the authors, who take full responsibility for the accuracy and integrity of the work.

Contents

독자와 범위	i
1 초록과 기여 경계	1
1.1 연구 질문	1
1.2 기여	2
1.3 한 문장 verdict	2
2 Operational definition: 은유가 아니라 causal boundary	3
2.1 엄격한 정의	3
2.2 Timing 과 medium 은 직교한다	4
2.3 포함과 제외	4
2.4 Sleep phase 와 biological stages	4
3 Training prerequisite map: 무엇이 실제로 바뀌는가	6
3.1 첫 용어 지도	6
3.2 Operator record	6
3.3 Training cost 는 FLOPs 만이 아니다	7
3.4 Wake, sleep, long-term memory 의 세 가지 계약	7
4 문제 분해: Sleep 은 무엇을 이기려는가	8
4.1 P1 – P4: static deployment, personalization, experience, freshness	8
4.2 P5 – P6: continual learning 과 bounded capacity	9
4.3 P7 – P8: latency isolation, deletion 과 rollback	9
4.4 P9 – P10: repeated long context 와 on-device adaptation	9
4.5 Synthetic replay 가 문제를 증폭할 때	10
5 계보: biological replay 에서 multi-rate neural memory 까지	11
5.1 1995: complementary learning systems	11
5.2 2017: continual-learning baselines 가 먼저 만든 consolidation 문제	11
5.3 2020 – 2022: NREM/REM analogy 를 계산 mechanism 으로 만든 연구	12
5.4 2023 – 2025: external hierarchy 와 wake-time neural memory	12
5.5 2026: 명시적 parametric sleep 과 product synthesis 의 분기	12

6	Alternative frontier: Sleep 이 없어도 가능한 최신 경로	14
6.1	Long context 와 retrieval	14
6.2	Recurrent/neural memory 와 serving cache	14
6.3	PEFT: 작은 parametric destination	14
6.4	Model editing: 좁은 update	15
6.5	Context compilation 과 one-forward adaptation	15
6.6	Self-adaptation 과 synthetic-data 위험	15
7	STC mechanisms 와 공개 system evidence	16
7.1	External-memory sleep: 현재 가장 성숙한 형태	16
7.2	Company evidence 를 과대 해석하지 않기	16
7.3	Parametric sleep: 직접적이지만 research-scale 인 증거	16
7.4	Hybrid promotion	17
8	Strongest-alternative matched comparison	18
8.1	비교 원칙	18
8.2	문제별 matched baseline	18
8.3	Hybrid recommendation	19
8.4	STC 가 이기지 못하는 영역	20
9	Evidence trajectory: 무엇이 실제로 성숙하고 있는가	21
9.1	Date-frozen corpus 와 재현 가능한 경계	21
9.2	Evidence maturity 와 expected value 는 다른 축이다	21
9.3	Industry 와 academia 의 서로 다른 수렴	21
9.4	번역 corpus 가 담당하는 역할	23
10	Promisingness: 새로운 축은 정말 최선인가	24
10.1	현재 증거가 지지하는 범위	24
10.2	여섯 개의 핵심 verdict	24
10.3	문제별 strongest alternative 와 sleep 의 조건	25
10.4	Workload 별 판단	25
10.5	의사결정 규칙	25
11	Mainstream scenarios: lifecycle 과 이름의 미래	28
11.1	다섯 개의 조건부 시나리오	28
11.2	현재 base case	29
11.3	분기점을 미리 보여 주는 signal	29
11.4	시나리오에 강건한 연구 전략	29

12 Scaling law: 법칙이 아니라 측정 가능한 가설부터	31
12.1 State 와 lifecycle work 를 먼저 달는 identity	31
12.2 Valid reuse 가 만드는 compile break-even	32
12.3 Finite state 가 만드는 capacity knee	34
12.4 Cadence 와 queue 의 내부 optimum	35
12.5 Fit candidate 와 법칙 승격 조건	35
13 System infrastructure: wake, sleep, memory 를 어떻게 잇는가	37
13.1 세 plane 과 하나의 publication transaction	37
13.2 Source-of-record 와 heterogeneous materialized views	37
13.3 Scheduler: operator, destination, cadence 를 분리한다	38
13.4 State migration roofline	38
13.5 네 가지 physical placement	40
13.6 운영 계약	40
14 Memory-device implication: 알고리즘 이름보다 workload primitive	41
14.1 왜 inference memory 와 다른 trace 가 생기는가	41
14.2 Cross-scenario workload classes	41
14.3 장치 요구사항을 metric 으로 번역하기	43
14.4 Device-system 공동 DSE	44
14.5 의사결정 순서	44
15 Falsifiable benchmark 와 experiment roadmap	45
15.1 독립 단위와 cycle	45
15.2 Observable 과 oracle 을 물리적으로 분리한다	45
15.3 Event 와 query family	46
15.4 Confirmatory methods 와 resource envelope	46
15.5 Metric stack	47
15.6 통계 · 재현성과 claim-down rule	47
16 Limitations, negative evidence 와 falsifiers	49
16.1 문헌 자체의 한계	49
16.2 Negative evidence 가 가리키는 구조적 위험	49
16.3 결론을 뒤집거나 좁힐 구체적 falsifier	50
16.4 Coverage 와 saturation 의 한계	51
16.5 본 논문이 주장하지 않는 것	51

17 Conclusion: 다음 학습 축의 조건과 연구 순서	52
17.1 처음의 질문에 대한 직접 답	52
17.2 권장 연구 순서	52
17.3 마지막 framing	53
Canonical claim – evidence ledger	54
참고문헌	58

List of Figures

2.1	Wake-capture-sleep-publication 의 causal lifecycle. 점선 경계 왼쪽은 현재 response 의 latency-critical path 이고, 오른쪽은 후속 wake 용 state 를 만드는 deferred path 다.	3
2.2	Update timing 과 memory medium 의 3×3 taxonomy. 같은 LoRA update 도 current answer 전에수행하면 test-time training, response 뒤에 수행하면 asynchronous consolidation 이다.	4
4.1	문제별 방법 적합도. 한 method 가 모든 row 에서 우세하지 않으며, external sleep 과 parametric sleep 도 서로 다른 failure surface 를 가진다.	8
5.1	1995 – 2026 evidence chronology. 연도는 공개 source 의 최초 publication/release 기준이며, 같은 시기의 방법이 동일 maturity 라는 뜻은 아니다.	11
8.1	External system-of-record 와 selective reversible parametric promotion. External evidence plane 은 authoritative 하고, parametric state 는 break-even 을 통과한 serving cache 로 취급한다.	19
9.1	Source freeze 시점의 industry-academia landscape. Industry 는 external synthesis 와 temporal provenance 에, academia 는 continual learning, neural memory, learned writer 와 capacity theory 에 더 넓게 분포한다. 좌표는 corpus 의 정성적 audit 를 시각화한 것이며 시장 규모가 아니다.	22
10.1	Evidence maturity 와 expected value 를 분리한 conditional verdict. 오른쪽 위로 갈수록 현재증거와 잠재 가치가 함께 높다. Parametric sleep 은 잠재 가치가 커도 성숙도가 낮은 연구 bet 로 남는다.	26
12.1	Scaling statement 의 네 등급. Identity 는 장부를 닫고, break-even 은 제한된 가정 아래경계를 계산하며, fit candidate 는 sweep 에 맞추고, hypothesis 는 반증 실험을 기다린다.	31
12.2	One-time sleep cost 와 future valid reuse 의 analytical crossover. Break-even 왼쪽 은 external retrieval 이, 오른쪽은 compiled artifact 가 유리할 수 있다. Quality-governance 차이가 있으면 경계가이동한다.	33
12.3	Finite-memory capacity knee 의 가설적 형태. Physical occupancy 가 한계에 닿기 전에 behavioral reachability 또는 plasticity 가 임계값 아래로 내려갈 수 있다. 곡선은 설명용이며 측정 결과가 아니다.	34
12.4	Cadence frontier. 너무 잦은 sleep 은 fixed overhead 와 interference 가 크고, 너무 늦은 sleep 은 staleness 와 queue age 가 크다. 내부 optimum 의 위치는 workload 와 resource vector 에 따라 변한다.	35
13.1	Wake/sleep placement 의 state-migration roofline. Arithmetic intensity 가 아니라 유용한 sleep work 대비 이동 state 의 비율과 link bandwidth 가 placement crossover 를 정한다. 표시된 영역은 설계공간이며 특정 GPU 의 측정 결과가 아니다.	39

14.1	공개 source 에서 관찰 가능한 workload class 를 evidence maturity 와 cross-scenario residual value 의 두 축에 배치한 research map. 좌표는 제품 추천이나 시장 예측이 아니라 benchmark 우선순위를정하기 위한 정성적 가설이다.	42
15.1	Falsifiable research agenda. Evidence stream, candidate construction, lifecycle validation, repeated-cycle evaluation, scaling fit 을 하나의 provenance chain 으로 연결한다.	45
16.1	Wake capture 에서 deletion 까지 이어지는 주요 failure mode 와 lifecycle control. 한 단계의 accuracy gate 만으로는 poison, stale publish, rollback resurrection, residual delete 를 막을 수 없다.	50

List of Tables

1.1	본 Study Paper 의 기여와 evidence boundary	2
2.1	본 연구의 포함·제외 판정	5
3.1	Study 에서 반복 사용할 training 개념과 Background 링크	6
3.2	대표 sleep operator 를 training record 로 펼친 예	7
4.1	문제별 가장 먼저 이겨야 할 baseline 과 STC 의 제한된 win condition	10
5.1	계보별 evidence 가 실제로 지지하는 범위	13
6.1	Alternative frontier 의 cost·capacity·governance 비교	15
7.1	External, parametric, hybrid sleep 의 운영 성질	17
8.1	문제별 strongest non-STC baseline 과 조건부 verdict	18
9.1	본문에서 사용하는 evidence maturity ladder	22
10.1	핵심 주장별 조건부 verdict 와 반증 조건	24
10.2	Workload 별 expected value 와 현재 evidence maturity	26
11.1	시나리오별 trigger, leading indicator, falsifier	28
11.2	Quarterly signal dashboard	29
13.1	Memory medium 별 serving path 와 rollback contract	38
13.2	Physical placement option 과 측정해야 할 trade-off	40
14.1	Device 연구가 먼저 계측할 workload primitive	43
15.1	독립적으로 변화시킬 event factor 와 주요 failure	46
15.2	한 결과표에 함께 있어야 할 metric stack	47
16.1	실패 mode 와 필요한 control	49
1	Veridraft-gated public claims (source freeze 2026-08-05)	54

초록과 기여 경계

초록

Persistent agent 가 wake 중 경험을 쌓고 응답 경로 밖에서 이를 재구성하기 시작하면서, 배포 후 학습을 pre-training/post-training 과 다른 축으로 볼 필요가 생겼다. 본 연구는 *sleep-time compute* 를 생물학적 은유가 아니라 “latency-critical response 뒤에 실행되어후속 wake episode 가 재사용할 persistent state 를 바꾸는 검증 가능한 변환” 으로 정의한다. Update destination 은 foundation weights 에 한정하지 않고 adapter, recurrent/neural state, text, vector, temporal graph 와 그 혼합을 포함한다.

2026-08-05 에 동결한 primary/official evidence 를 기반으로 세 질문을 분리한다. 첫째, 새로운 lifecycle 축이 실제로 등장했는가? 둘째, 그 축 안에서 parametric training 이 external-memory synthesis 보다우세한가? 셋째, 어느 scenario 에서도 살아남는 system-memory-device primitive 는 무엇인가? 답은비대칭적이다. Background external-memory synthesis 는 공개 production system 에서 관찰되지만, per-user foundation-weight sleep 은 continual retention, deletion, rollback, security, fleet economics 를동시에 입증하지 못했다. 따라서 near-term default 는 external source-of-record 에 evidence 와 provenance 를유지하고, high-reuse-stable pattern 만 reversible adapter 또는 bounded parametric cache 로 승격하는 hybrid 다.

본 논문은 sleep 이 풀려는 문제를 static deployment, personalization, agent experience, knowledge freshness, continual learning, bounded capacity, latency isolation, deletion/rollback, repeated long context, on-device adaptation 으로 분해한다. 각 문제에 대해 long context, RAG, external memory, model editing, continual-learning regularization/replay, global periodic refresh 라는 strongest non-STC baseline 을 먼저세운다. 이 비교에서 sleep timing 은 freshness·batching·validation 격리를 제공하지만, capacity 와 governance 문제를 자동으로 해결하지 않으며 one-off context 에는 대체로 이점이 없다.

Scaling 분석은 네 층으로 제한한다. 실제 trace 에서 측정할 identity, reuse break-even model, cadence 와 capacity-knee fit candidate, 그리고 “valid reusable evidence per sleep cost” 같은 falsifiable hypothesis 다. 마지막으로 state movement, snapshot/COW, high-endurance index/graph tier, provenance filtering, adapter fabric, pooled memory, secure delete/rebuild, neural-state caching, on-device consolidation 을 scenario robustness 에 따라 우선순위화한다.

1.1 연구 질문

1. **Existence:** sleep-time compute 는 기존 offline maintenance 의 새 이름인가, 독립된 lifecycle-system boundary 인가?
2. **Necessity:** long context, retrieval, editing, continual learning, global refresh 가 있는데어떤 workload 에서 deferred consolidation 이 추가로 필요한가?
3. **Medium:** persistent knowledge 를 weights/adapter 에 넣을지, text/vector/graph 로 둘지, 어떤 hybrid promotion rule 이 최선인가?
4. **Capacity:** session 이 누적될 때 forgetting, interference, stale memory, delete debt, recursive synthetic data 를 어떻게 bounded 하게 만드는가?

5. **Scaling**: sleep FLOPs 가 아니라 무엇을 독립변수와 outcome 으로 측정해야 하는가?
6. **Infrastructure**: wake inference/training, sleep transformation, persistent memory 가 state movement 와 publication 을 최소화하도록 어떻게 배치되어야 하는가?

1.2 기여

Table 1.1: 본 Study Paper 의 기여와 evidence boundary

기여	산출물	주장 경계
정의 분류	causal lifecycle definition 과 제외 규칙 timing × medium × scope × persistence taxonomy	생물학적 동일성은 주장하지 않음 architecture 이름을 evidence 로 쓰지 않음
대안 비교	문제별 strongest baseline 과 non-win region	straw-man baseline 금지
근거 평가	maturity ≠ expected value matrix, negative evidence	검색 포화를 주장하지 않음
scaling	identity, break-even, candidate surface, falsifier	universal law 로 부르지 않음
system/device	state-flow architecture 와 opportunity stack	시장 규모 · 매출 예측이 아님
재현성	57-claim ledger, 73-source registry, 15 figure ledger	date-frozen snapshot

1.3 한 문장 verdict

핵심 조건부 verdict

Sleep-time compute 는 **background transformation lifecycle** 로서는 이미 유망하고 일부는 production 이지만, **broad personalized parametric training** 으로서는 아직 미성숙하다. 앞으로의 주류 형태는 “모델이 밤마다 전체 weights 를 다시 배운다” 보다 external memory maintenance 와 selective parametric promotion 을 포괄하는 control plane 일 가능성이 높다.

이 결론은 이름의 유행과 분리한다. Industry 가 dreaming, reflection, synthesis, self-evolution, memory maintenance 라고 부르더라도 causal boundary 가 같으면 같은 분석 대상이다. 반대로 “sleep” 이라는 단어를 써도 current response 계산이나 passive cache eviction 만 수행하면 본 정의에는 들어오지 않는다.

Operational definition: 은유가 아니라 causal boundary

2.1 엄격한 정의

본 연구에서 sleep-time compute 는 다음 다섯 조건을 모두 만족하는 computation 이다.

1. **Wake-derived input:** 실제 interaction, feedback, tool trace, retrieved memory, model failure 또는 그에 연결된 evidence 를 입력으로 삼는다.
2. **Deferred causal boundary:** 현재 response 의 latency-critical dependency 가 끝난 뒤 실행되며 현재 답을 완성하기 위한 필수 compute 가 아니다.
3. **Reusable destination:** 후속 session/query 가 읽을 persistent 또는 bounded-lifetime state 를 바꾼다.
4. **Transformation:** append 만 하지 않고 select, summarize, link, replay, train, edit, distill, compress, merge, evict 중 하나 이상을 수행한다.
5. **Validation/publication:** 후보 state 와 serving state 의 경계를 두고 실패 시 reject 또는 rollback 할 수 있다.

즉 sleep-time compute 는 latency-critical response 뒤로 연기되고, 다음 wake episode 가 재사용할 state 를 바꾸는 lifecycle 이다. [STC-C005] 이 정의는 Model Needs Sleep 의 post-response consolidation, OpenAI Dreaming 의 background synthesis, Mem0 Dream 의 scheduled transformation 에 공통으로 적용된다 (Lin et al. 2025; OpenAI 2026; Mem0 2026a).

Wake-test-sleep lifecycle

경계는 clock time이 아니라 response-critical dependency다

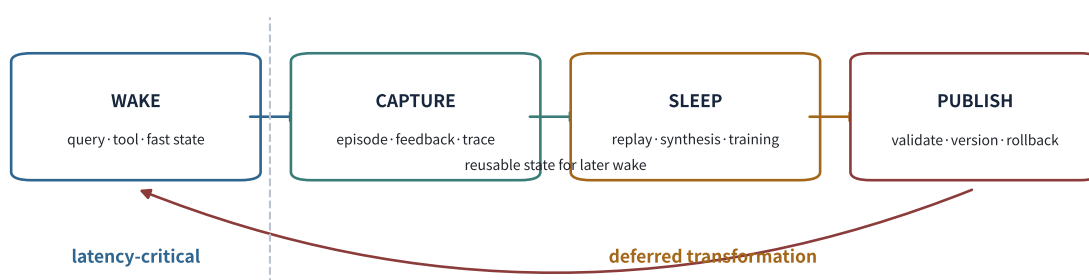


Figure 2.1: Wake–capture–sleep–publication 의 causal lifecycle. 점선 경계 왼쪽은 현재 response 의 latency-critical path 이고, 오른쪽은 후속 wake 용 state 를 만드는 deferred path 다.

[STC-F001]

출처/제작: Original synthesis; SRC-STC-0014, SRC-STC-0052 대체 설명: Wake, capture, sleep, publish 네 상자가 순서대로 연결되고 검증된 재사용 상태가 다음 wake 로 되돌아오는 순환도.

2.2 Timing 과 medium 은 직교한다

Update timing 과 memory medium 은 독립 축이다. Sleep phase 는 weights, adapters, recurrent state, text/vector store, temporal graph 또는 mixture 를 바꿀 수 있다.^[STC-C006] 따라서 “weights 를 바꾸지 않으면 training 이 아니므로 sleep 도 아니다” 와 “background 에서 실행되면 모두 sleep 이다” 는 둘 다 너무 좁거나 넓다.

두 개의 독립 축: update timing × memory medium

	Within-query	Post-response	Periodic sleep
Ephemeral / state	KV / recurrent state	session fast state	state compaction
External text · vector · graph	retrieval write	Letta / Mem0	Dreaming / graph rebuild
Parametric weights · adapter	TTT / neural memory	async adapter	replay / distill / refresh

같은 medium도 timing이 다르면 SLA · data horizon · validation이 달라진다

Figure 2.2: Update timing 과 memory medium 의 3×3 taxonomy. 같은 LoRA update 도 current answer 전에 수행하면 test-time training, response 뒤에 수행하면 asynchronous consolidation 이다.
[STC-F002]

출처/제작: Original synthesis; SRC-STC-0014, SRC-STC-0021, SRC-STC-0016 대체 설명: 열은 query 내부, response 이후, 주기적 sleep 이고 행은 ephemeral state, 외부 text-vector-graph, parametric weight-adapter 인 격자.

분류를 위해 각 operator 를 6-tuple 로 기록한다.

$$O = (t_{trigger}, D_{horizon}, M_{source}, M_{dest}, S_{scope}, P_{publish}), \quad (2.1)$$

여기서 trigger 는 언제 시작하는지, $D_{horizon}$ 은 어느 기간 data 를 보는지, source/destination medium 은 무엇인지, scope 는 query/session/user/cohort/global 중 어디인지, publication policy 는 어떻게 후보를 활성화하는지 나타낸다. 같은 architecture 라도 이 tuple 이 다르면 system workload 가 다르다.

2.3 포함과 제외

2.4 Sleep phase 와 biological stages

NREM/REM 은 replay 와 perturbation, integration 과 abstraction 을 생각하게 하는 productive analogy 다. 그러나 구현은 EEG stage 를 모방할 필요가 없다. 본 연구는 기능을 다음처럼 매핑한다.

- NREM-like: high-fidelity replay, recent episode integration, stable fact linking,
- REM-like: perturbed/generative replay, abstraction, counterfactual strategy discovery,

Table 2.1: 본 연구의 포함 · 제외 판정

행동	판정	이유
현재 query 의 long chain-of-thought	제외	persistent reusable state 를 바꾸지 않음
token-by-token TTT 후 현재 답 생성	wake/test-time	현재 response dependency 안에 있음
post-response memory agent 가 profile 수정	포함	deferred transformation + later reuse
nightly vector re-embedding/index rebuild	포함	external reusable state 변환
LRU 가 cache line 을 passive eviction	제외	learned/validated consolidation 이 아님
주기적 global continued pre-training	조건부 포함	wake-derived stream 과 explicit later-use lifecycle 이면 포함
배포 전 generic offline training	제외	wake experience 와 persistent deployed loop 가 없음
검증 없는 background summary append	미완성	transformation 은 있지만 publication/gate 가 부재

- publication: biological analogy 에는 약하지만 production system 에는 필수인 validation·roll-back.

마지막 항이 중요한 이유는 agent memory 가 사용자 data 와 serving artifact 를 다루기 때문이다. Biology 는 배울 mechanism 을 제공하지만 tenant scope, deletion SLA, source license, atomic rollout 을 해결하지 않는다.

주의 Phase 와 mechanism 을 혼동하지 말 것

Sleep 은 언제와 어떤 *lifecycle* 에서 변환하는지, replay·distillation·RL·editing 은 어떻게 변환하는지, weights/text/vector/graph 는 어디에 저장하는지를 답한다. 세 질문을 분리해야 동일 resource 에서 matched baseline 을 만들 수 있다.

Training prerequisite map: 무엇이 실제로 바뀌는가

Sleep-time compute 를 평가하려면 “learning” 을 하나의 동사로 쓰지 않고 data, objective, trainable state, optimizer, validation, serving artifact 로 분해해야 한다. 이 장은 별도 Training Background 와 Study 사이의 navigation layer 다.

3.1 첫 용어 지도

Table 3.1: Study 에서 반복 사용할 training 개념과 Background 링크

개념	최소 의미	STC 에서 묻는 질문
Fine-tuning ^[배경] / continued pre-training ^[배경]	좁은 목적 또는 추가 corpus 로 배포 모델을 갱신	base 전체인가, delta 인가?
LoRA ^[배경] / adapter ^[배경]	작은 trainable state 로 behavior 를 조정	user/domain 별 artifact 수와 merge 는?
Distillation ^[배경]	teacher behavior 를 student 로 압축	tail·uncertainty·provenance 가 보존 되는가?
Replay buffer ^[배경] / augmentation ^[배경]	과거 · 변형 data 를 new data 와 섞음	retention 과 recursive-data depth 는?
Continual learning ^[배경]	stream 을 배우며 이전 능력을 유지	multi-cycle forgetting 과 capacity knee 는?
RLHF/RL ^[배경]	delayed/preference signal 로 policy 를 조정	model 이 아니라 trigger/router 를 학습할 것인가?
Meta-learning ^[배경]	빠른 adaptation 이 가능하도록 학습	user sleep inner loop 가 안정적인가?
Test-time training ^[배경]	test stream 에서 state 를 즉시 갱신	current response path 인가 deferred path 인가?
Model editing ^[배경] / un-learning ^[배경]	좁은 수정 또는 영향 제거	locality, lineage, rollback 단위는?

3.2 Operator record

각 연구 방법을 다음 record 로 정규화한다.

$$R_{op} = \{D, T, \theta_{train}, \mathcal{L}/r, U, C, V, A\}, \quad (3.1)$$

D 는 wake-derived data 와 replay, T 는 teacher/source, θ_{train} 은 바뀌는 state, $\mathcal{L}/r$ 은 loss 또는 reward, U 는 update rule/optimizer, C 는 capacity destination, V 는 validation slices, A 는 published artifact 와 rollback unit 이다. 이 record 가 비어 있으면 “모델이 기억을 공고화했다” 는 설명을 system 으로 환원할 수 없다.

Table 3.2: 대표 sleep operator 를 training record 로 펼친 예

Operator	Data/teacher	Trainable state	Objective/update	Capacity	Artifact/gate
summary synthesis	episodes, LLM teacher	없음 또는 prompt/model	fidelity, utility, compression	external text	versioned record, QA
temporal graph	event+time+entity	graph nodes/edges	merge, supersede, conflict	external graph	graph epoch, traversal test
LoRA sleep	new+replay	low-rank A, B	task+retention loss, Adam	adapter bank	candidate delta, shadow
distillation	teacher puts+real anchor	student/base delta	CE+KL+retention	model artifact	full release suite
RL controller	trajectories	controller ϕ	delayed utility-cost reward	policy model	constrained canary
model edit	fact+neighbors	localized weight delta	efficacy+locality	parametric patch	edit version, rollback

3.3 Training cost 는 FLOPs 만이 아니다

Full fine-tuning 은 weights 외에 gradient, optimizer state, activation 을 필요로 한다. LoRA 는 trainable parameter 와 optimizer bytes 를 줄이지만 base read, forward/backward activation, validation, version management 는 남는다. External transformation 은 GPU training 이 없을 수 있지만 teacher inference, embedding, graph write, dedup, index rebuild 와 evaluation 을 낸다.

따라서 end-to-end sleep cost 를

$$C_{sleep} = C_{ingest} + C_{transform} + C_{train} + C_{validate} + C_{publish} + C_{retain} \quad (3.2)$$

로 분리한다. C_{retain} 은 artifact 가 존재하는 동안의 storage, migration, rebase, deletion debt 다. Kernel FLOPs 만 비교하면 external path 의 retrieval cost 와 parametric path 의 lifecycle cost 를 서로 다른 회계 기준으로 비교하게 된다.

3.4 Wake, sleep, long-term memory 의 세 가지 계약

Wake path 는 response SLA 와 isolation 을, sleep path 는 throughput·deadline·candidate safety 를, long-term memory plane 은 persistence·scope·version·delete 를 계약한다. 한 cluster 에서 모두 실행할 수도 있지만 logical contract 는 분리한다. Shared state 는 explicit snapshot epoch 를 가져야 wake 가 training 중간값을 읽지 않는다.

핵심 논문을 읽는 최소 checklist

Data 는 어디서 왔는가? 어느 state 가 얼마나 오래 바뀌는가? Old/new/safety 를 무엇으로 검증했는가? Capacity 가 찰 때 무슨 state 를 제거하는가? 후보와 serving state 를 어떻게 전환하는가? 이 다섯 답이 없으면 accuracy gain 을 production sleep system 으로 해석하지 않는다.

문제 분해: Sleep 은 무엇을 이기려는가

새 축의 가치는 이름이 아니라 기존 방법이 남긴 residual problem 에서 나온다. 이 장은 sleep-time compute 가 겨냥하는 열 가지 문제를 strongest current default 와 함께 놓는다.

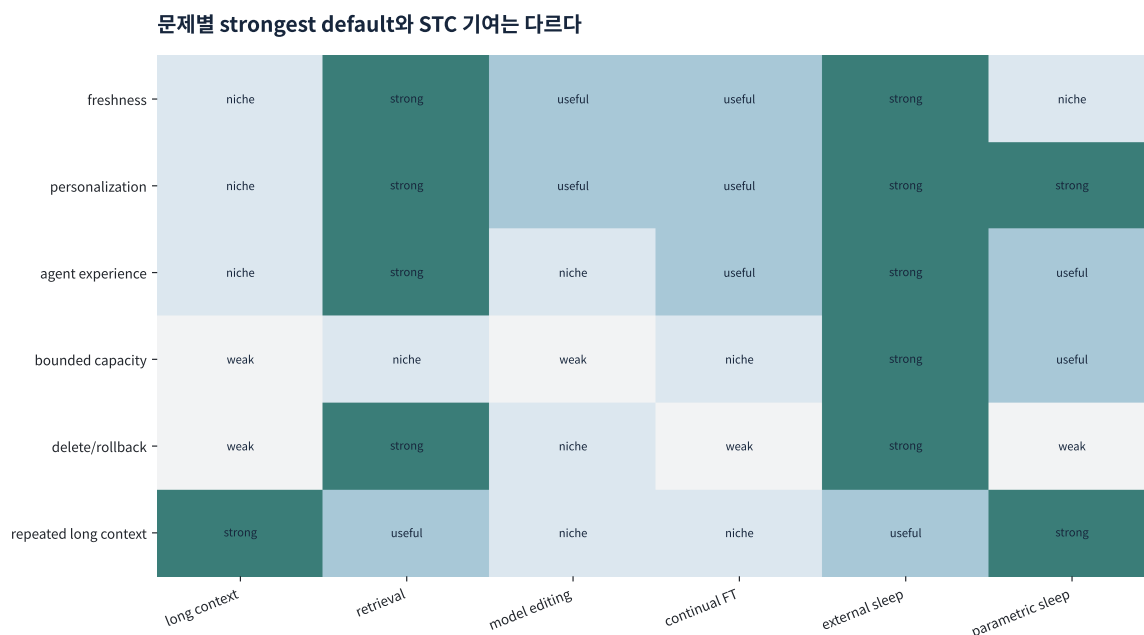


Figure 4.1: 문제별 방법 적합도. 한 method 가 모든 row 에서 우세하지 않으며, external sleep 과 parametric sleep 도 서로 다른 failure surface 를 가진다.

[STC-F004]

출처/제작: Original synthesis; SRC-STC-0002, SRC-STC-0031, SRC-STC-0036 대체 설명: 행은 freshness, personalization, agent experience, bounded capacity, deletion, repeated long context 이고 열은 long context, retrieval, editing, continual FT, external sleep, parametric sleep 인 색상 행렬.

4.1 P1 – P4: static deployment, personalization, experience, freshness

Static deployment. Pre/post-training 이 끝난 fixed model 은 배포 뒤 새 사용자 분포와 도구 변화에 자동 적응하지 않는다. Strongest default 는 prompt/profile retrieval 과 주기적 global refresh 다. Sleep 은 반복되는 local distribution 을 idle window 에서 정리해 다음 query 의 retrieval 또는 adapter 를 미리 만드는 데 기여한다. One-off interaction 이면 amortization 이 없다.

Personalization. External profile 은 explicit, editable, portable 하므로 첫 baseline 이다. Parametric sleep 은 같은 preference 가 매우 자주 재사용되고 prompt token-retrieval latency 가 누적될 때 후보가 된다. 다른 user 로의 leakage 와 preference drift 가 크면 isolated external record 가 우세하다.

Agent experience. Agent 는 성공 · 실패 trace 에서 strategy 를 추출할 수 있다. Raw transcript 를 매번 long context 로 넣는 대신 external strategy bank 나 procedural graph 로 압축하는 것이 강한 default 다. Skill 이 stable 하고 반복되면 distillation 이나 adapter promotion 을 비교한다.

Knowledge freshness. Mutable fact 는 timestamp, supersession, citation 이 필요한 external temporal memory 가 기본이다. Weight refresh 는 stale fact 를 지우기 어렵고 old/new fact 가 composition 에서 충돌할 수 있다. 실제로 sequential fact 가 weights 에 encoding 되어도 behaviorally unreachable 할 수 있으므로 retention 은 parameter storage 가 아니라 access 와 composition 으로 측정해야 한다.^[STC-C032] (O'Neill 2026)

4.2 P5 – P6: continual learning 과 bounded capacity

Continual learning 은 replay, regularization, gradient constraint, isolation, growth/compaction 의 도움을받지만 어느 것도 무한 retention 을 보장하지 않는다. Stable skill/procedure 에는 parametric integration 이 유망하고, exact changing fact 와 legal delete 대상에는 external representation 이 유리하다.

정적 memorization capacity 실험은 sequential access, interference, governance 를 포함하지 않으므로 safe continual-learning capacity 를 직접 결정하지 못한다.^[STC-C033] (Morris et al. 2025; O'Neill 2026) Capacity 는 weights 의 bits 뿐 아니라 replay bytes, adapter count, router fragmentation, validation time, index latency, rollback versions 의 vector budget 이다.

Rate-distortion 은 무엇을 어느 fidelity 로 압축할지 utility-budget trade-off 로 만드는 유용한 관점이지만, 제안된 frontier 가 아직 agent 의 보편 empirical scaling law 는 아니다.^[STC-C034] (Colaco and Lahjouji 2026) Future-query distortion 을 reconstruction error 하나로 대체하지 말고 task utility, provenance recovery, temporal correctness, delete granularity 를 포함한다.

4.3 P7 – P8: latency isolation, deletion 과 rollback

Sleep 의 가장 강한 system-level 이유는 expensive transformation 을 current response SLA 에서 분리하는것이다. 그렇다고 반드시 별도 datacenter cluster 가 필요한 것은 아니다. State 가 있는 device/region 에서 local sleep 을 실행하면 migration 을 줄일 수 있고, shared accelerator pool 이 작은 job 을 batch 할 수도 있다.

Deletion 은 source episode 가 어떤 summary, vector, graph edge, training dataset, adapter 에 기여했는지 lineage 를 요구한다. External memory 는 더 직접적이지만 poisoned record 가 agent behavior 를 조용히조종할 수 있는 attack surface 를 만든다. 따라서 provenance 와 revocation 은 부가 metadata 가 아니라설계 요구다.^[STC-C037] (Chen et al. 2024b)

4.4 P9 – P10: repeated long context 와 on-device adaptation

Long context 는 원문에 대한 one-off deep reasoning 에서 강하다. Sleep compilation 은 context 를 summary, graph, latent/parametric state 로 바꾸는 선행 비용이 있으므로 future query distribution 이 예측 가능하고 reuse 가 충분할 때만 이긴다. Long context 를 대체하기보다 build path 로 사용하고 compact state 를 serve path 에서 쓰는 complement 다.

On-device sleep 은 privacy, intermittent connectivity, local personalization 에서 expected value 가 높지만 thermal, battery, write endurance, limited validation compute 가 제약이다. Cloud 와 device 사이에서 raw episode 대신 provenance-preserving delta, gradient sketch, summary 만 이 동시키는 split 도 비교 대상이다.

4.5 Synthetic replay 가 문제를 증폭할 때

Generated replay 를 costless fresh data 로 볼 수 없다. Real-data support 가 사라진 recursive training 은 distribution tail 을 collapse 시킬 수 있다.^[STC-C035] (Unknown organization 2024) 반대로 real 과 synthetic data 를 적절히 누적한 조건에서 collapse 는 필연이 아니므로, 중요한 변수는 synthetic 여부의 이분법보다 recursive depth 와 real-data anchoring 이다.^[STC-C036] (Gerstgrasser et al. 2024)

Table 4.1: 문제별 가장 먼저 이겨야 할 baseline 과 STC 의 제한된 win condition

문제	Strongest default	STC win condition
freshness	temporal retrieval	반복 query 와 controlled synthesis
personalization	external profile	stable high reuse, reversible scope
agent experience	strategy memory/reflection	multi-episode compression improves later utility
continual skill capacity	replay/PEFT/global refresh admit/evict/compress	cycle-level retention at bounded state marginal utility exceeds compaction/deletion cost
long context deletion	long-context inference/RAG external lineage + rebuild	compile cost amortized over valid reuses parametric state remains reconstructable cache

계보: biological replay에서 multi-rate neural memory까지

Sleep-time compute 는 2026 년에 갑자기 생긴 단일 논문 계열이 아니다. Fast/slow learning theory, continual learning, generative replay, external agent memory, test-time neural memory, asynchronous memory agents 가 서로 다른 문제를 풀다가 하나의 lifecycle 로 수렴한다.

1995–2026: 이름보다 먼저 존재한 계보



Figure 5.1: 1995 – 2026 evidence chronology. 연도는 공개 source 의 최초 publication/release 기준이며, 같은 시기의 방법이 동일 maturity 라는 뜻은 아니다.

[STC-F003]

출처/제작: Original synthesis; SRC-STC-0001–SRC-STC-0063 대체 설명: 1995 CLS 에서 2017 continual learning, 2020 PAD, 2023 MemGPT, 2025 Titans/HOPE, 2026 parametric sleep 과 product dreaming 으로 이어지는 시간축.

5.1 1995: complementary learning systems

CLS 는 hippocampus 의 빠른 episodic acquisition 과 neocortex 의 느린 interleaved learning 을 분리했다 (McClelland et al. 1995). 이 관점이 제공하는 basis 는 “새 기억을 빠르게 받는 store” 와 “기존 구조를 보존하며 천천히 통합하는 learner” 를 한 update rule 로 강제하지 않는다는 점이다. 오늘날의 system 으로 번역하면 wake log/episodic memory 와 sleep consolidation model 의 역할 분리다.

그러나 biology 에서 production architecture 를 곧바로 도출할 수는 없다. Biological model 은 user scope, exact delete, source attribution, model release compatibility 를 다루지 않는다. 따라서 계보는 mechanism inspiration 이지 deployment evidence 가 아니다.

5.2 2017: continual-learning baselines 가 먼저 만든 consolidation 문제

Deep Generative Replay 는 generator 의 pseudo-data 와 새 data 를 함께 학습해 과거 task forgetting 을 줄이는 강한 non-sleep-named baseline 이다. [STC-C019] (Shin et al. 2017) EWC 는 이전 task 에 중요한 parameter 를 Fisher-weighted quadratic penalty 로 보호해 stability 와 plasticity 를 trade-off 한다. [STC-C020] (Kirkpatrick et al. 2016) GEM 은 stored examples 의 gradient 를 constraint 로

사용해 이전 task loss 가 국소적으로 증가하지 않게 update 를 투영한다.[STC-C021] (Lopez-Paz and Ranzato 2017)

Progress & Compress 는 active column 에서 새 task 를 배우고 consolidation phase 에서 knowledge base 로 압축하는 periodic-refresh 대안을 제시했다.[STC-C022] (Schwarz et al. 2018) 이 계열은 중요한 교훈을 준다. “Sleep” 이라는 이름 없이도 progress/online phase 와 compress/offline phase 를 나눌 수 있으며, 새 term 의 가치는 matched resource 에서 기존 periodic consolidation 보다 나아야 한다.

5.3 2020 – 2022: NREM/REM analogy 를 계산 mechanism 으로 만든 연구

PAD(Perturbed and Adversarial Dreaming) 는 wake, NREM-like perturbed replay, REM-like adversarial dreaming 을 분리해 cortical representation learning 을 연구했다.[STC-C018] (Deperrois et al. 2022) Hippocampus–neocortex computational model 은 autonomous offline dynamics 와 alternating NREM/REM regime 로 새 정보 통합과 old knowledge 보호를 함께 탐구했다.[STC-C017] (Singh et al. 2022)

2022 년 PLOS Computational Biology 의 spiking-network 연구는 두 task 설정에서 interleaved sleep-like replay 가 joint synaptic representation 을 형성하고 catastrophic forgetting 을 줄였다고 보고했다. [STC-C016] (Golden et al. 2022) 이것은 user-scale LLM deployment 증거가 아니라, online data 없이 autonomous replay 가 consolidation 을 도울 수 있다는 mechanism evidence 다. Task 수, modality, network scale, governance 의 외삽을 분리해야 한다.

5.4 2023 – 2025: external hierarchy 와 wake-time neural memory

MemGPT 는 제한된 context 와 external storage 사이에서 information 을 paging 하는 OS-style hierarchy 로 long-term agent memory 를 framing 했다.[STC-C011] (Packer et al. 2023) 여기서 중요한 변화는 “모델이 모든 것을 weights 에 기억해야 한다” 에서 “virtual memory 처럼 context 와 archival memory 를 관리한다” 로의 이동이다.

Titans 는 surprise-driven update 로 test time 에 neural memory state 를 학습하지만, 그 자체는 wake-path fast-state learning 이지 deferred cross-session sleep 은 아니다.[STC-C023] (Behrouz et al. 2024) 이 구분은 user framing 의 핵심이다. Test-time learning 은 current/near-current stream 에 적응하고, sleep-time learning 은 더 넓은 replay 와 검증을 거쳐 longer-lived state 를 publish 한다.

Nested Learning/HOPE 는 optimization process 를 여러 update frequency 의 nested memory level 로 보아 cadence 를 분석하는 유용한 틀을 제공하지만, 공개 evidence 는 deployed asynchronous sleep consolidation 을 입증하지 않는다.[STC-C024] (Behrouz et al. 2025) HOPE 의 CMS 처럼 서로 다른 frequency 로 갱신되는 state 는 wake/sleep scheduler 에 자연스럽게 매핑되지만, frequency hierarchy 와 lifecycle publication 은 별개 요구다.

5.5 2026: 명시적 parametric sleep 과 product synthesis 의 분기

2026 년 계보는 둘로 갈라진다. 한쪽은 Language Models Need Sleep 처럼 self-modification 과 replay-like training 으로 parametric memory 를 통합하는 연구다. 다른 쪽은 Dreaming, Mem0 Dream, Letta 처럼 external memory 를 background 에서 합성하는 product lifecycle 이다. 두 방향은 같은 sleep boundary 를 공유하지만 state medium 과 evidence maturity 가 다르다.

Table 5.1: 계보별 evidence 가 실제로 지지하는 범위

계보	지지하는 것	아직 지지하지 않는 것
biological/computational sleep	autonomous replay 와 multi-phase integration 가능성	LLM fleet economics, delete/rollback
continual learning	forgetting 을 줄 이 는 replay-constraint-isolation mechanism	무한 capacity, task-free production stability
external agent memory	bounded context 밖 persistent state 와 hierarchy	parametric skill acquisition
neural/test-time memory	forward path 에서 learned state update 가능성	deferred multi-session publication
explicit parametric sleep	self-modification+consolidation research proof	broad per-user production deployment
product dreaming	scheduled external background synthesis	foundation-weight training 공개 증거

핵심 계보의 종합

새로운 것은 replay 자체가 아니라, persistent agent product 가 wake-derived state 를 background 에서 계속변환하며 serving lifecycle 로 운영하기 시작했다는 점이다. 과학적 novelty 는 matched alternative 대비효과와 scaling surface 에서, 산업적 novelty 는 publication-roll-back-memory plane 의 지속 운영에서 평가한다.

Alternative frontier: Sleep 이 없어도 가능한 최신 경로

Sleep-time compute 을 promising 하다고 판단하려면 이미 강한 방법을 약하게 묘사하면 안 된다. 이 장은 long context, retrieval, model editing, PEFT, context compilation, self-adaptation, recurrent memory caching 을 independent alternative 또는 component 로 평가한다.

6.1 Long context 와 retrieval

Long context 는 원문을 손실 없이 한 번에 읽어야 하는 task 의 default 다. Context window 와 attention 효율이 개선되면 sleep compilation 의 일부 필요가 줄어든다. 그러나 매 query 마다 같은 long history 를 읽으면 token·KV·prefill cost 가 반복되고, context selection 과 stale conflict 가 남는다. Sleep 은 repeated workload 에서 hierarchy/summary/index 를 만드는 build-time complement 다.

Retrieval-Augmented Generation 은 mutable knowledge 를 weights 밖에 두고 retrieved passage 로 generation 을 condition 하므로 update cost 와 governance 의 강한 baseline 이다.^[STC-C026] (Lewis et al. 2020) Record-level source, time, delete 가 중요한 fact 에서는 parametric sleep 이 retrieval 을 이기기 전에 정확도뿐 아니라 rollback 과 rebase cost 까지 이겨야 한다.

6.2 Recurrent/neural memory 와 serving cache

RNN-style 또는 neural-memory model 은 fixed-size state 로 long history 를 압축해 attention 의 context cost 를 낮출 수 있다. 그러나 prefix KV cache 처럼 많은 request 가 같은 reusable state 를 공유하려면 state 를 materialize 하고 key 를 정하며 cache/evict/share 해야 한다. Memory Caching 은 이 serving 문제를 명시한다. ^[STC-C025] (Behrouz et al. 2026b)

이는 sleep 과 보완적이다. Wake model 이 만드는 recurrent state 를 session boundary 에서 저장하고, sleep 이 state 를 compact/merge/rekey 할 수 있다. 반대로 state reuse 가 낮고 매 session 이 unique 하면 cache tier 가 추가 complexity 만 만든다. 평가에는 cache hit, state bytes, recompute cost, compatibility hash 가 필요하다.

6.3 PEFT: 작은 parametric destination

LoRA 는 adaptation 을 low-rank weight delta 로 제한해 trainable parameter 와 optimizer-state cost 를 줄이지만, sequential interference 와 lifecycle governance 를 없애지는 않는다.^[STC-C027] (Hu et al. 2021; O'Neill 2026) User 별 adapter 가 작아도 수억 개가 되면 metadata lookup, base-version compatibility, batching fragmentation, merge/delete 가 새 serving problem 이다.

PEFT 는 broad full-weight sleep 보다 유망한 parametric destination 이다. Base 를 immutable 하게 두고 candidate adapter 를 version/disable 하기 쉬우며, high-reuse behavior 를 external profile 보다 적은 wake overhead 로 serve 할 수 있다. 하지만 stable fact 를 adapter 에 넣으면 provenance 와 temporal update 가 어려워진다.

6.4 Model editing: 좁은 update

ROME 은 transformer MLP 의 factual association 을 localized edit 하여 targeted parametric change 가 가능함을보였지만 stable unbounded continual accumulation 을 입증한 것은 아니다.[STC-C028] (Meng et al. 2022a) MEMIT 은 많은 memories 에 대한 mass editing 을 확장했지만, 여전히 bounded batch intervention 이지 lifelong sleep loop 의 증거는 아니다.[STC-C029] (Meng et al. 2022b)

Editing 은 correction latency 가 중요하고 edit set 이 작을 때 strong baseline 이다. Sleep replay 는 여러 episode 에서 stable pattern 을 통합할 때 strong 하다. 양쪽 모두 efficacy 외에 paraphrase, locality, old edit retention, multi-hop composition, rollback 을 측정해야 한다.

6.5 Context compilation 과 one-forward adaptation

Generative Adapter 는 context 를 한 forward pass 로 parameter-like adapter 에 compile 하는 latent-memory 경로를보인다. 그 가치는 이후 reuse 가 compilation 과 version management cost 를 생각할 때 생긴다. [STC-C030] (Chen et al. 2024a) Gradient-based sleep 보다 빠를 수 있으나 teacher/compiler quality, base-version drift, adapter capacity, delete lineage 가 남는다.

이 접근은 long context 와 parametric memory 의 중간점이다. 원문을 build path 에서 한 번 읽고 compact state 로 serve 한다. One-off document 에는 compile 하지 않고, predicted query count 가 break-even 을 넘는 document/user cluster 만 선택하는 policy 가 필요하다.

6.6 Self-adaptation 과 synthetic-data 위험

Self-Adapting Language Models 는 model 이 adaptation data 와 update rule 을 스스로 생성해 wake learning 과 later consolidation 을 잊지만 recursive data quality risk 를 만든다.[STC-C031] (Zweiger et al. 2025; Unknown organization 2024) Self-generated curriculum 이 novelty 를 늘리는지, 같은 error 를 증폭하는지, real evidence 와 얼마나 연결되는지 cycle depth 별로 평가한다.

Table 6.1: Alternative frontier 의 cost-capacity-governance 비교

방법	One-time cost	Per-query cost	Capacity behavior	주요 failure
long context	낮음	token/KV/prefill 반복	context window 와 selection	distractor, latency
RAG/external	ingest/index	retrieval+context	independent storage growth	miss, poisoning, stale index
recurrent state	state build	small state forward	fixed/compressed state	state collision, low reuse
LoRA/adapter	training+validation	routing+delta compute	artifact bank 증가	interference, fragmentation
model editing	locate/edit/test	거의 없음	edits 누적	locality, composition
context compile	compiler inference	adapter reuse	compiled artifact 증가	break-even, rebase
global refresh	large periodic train	model-only inference	release 단위 재압축	freshness lag, high cost

핵심 Alternative 와 STC 의 관계

Sleep 은 별도 optimizer family 가 아니다. Retrieval index rebuild, recurrent-state compaction, adapter training, model editing, distillation, global refresh 를 언제 · 어떤 data horizon 에서 실행하고 어떻게 publish 할지 묶는 lifecycle 다. 따라서 “STC vs RAG” 보다 “RAG-only vs background synthesis vs selective promotion” 같은 system configuration 을 비교해야 한다.

STC mechanisms와 공개 system evidence

이 장은 실제 system 을 external synthesis, parametric consolidation, hybrid promotion 으로 나누고, 공개 evidence 가 무엇을 말하는지와 말하지 않는지를 분리한다.

7.1 External-memory sleep: 현재 가장 성숙한 형태

OpenAI Dreaming 은 external user memory 를 background 에서 종합하는 production feature 이며, 공개 설명은 per-user foundation-weight training 을 말하지 않는다.^[STC-C007] (OpenAI 2026) 이는 memory item 수를 그대로 늘리는 대신 관계와 장기 pattern 을 만들어 다음 interaction 을 돕는다는 점에서 strict lifecycle 정의에 들어간다.

Mem0 Dream 은 scheduled background synthesis 와 provenance-preserving additive output 을 제공하며, merge 와 supersede 는 ingest-time operation 으로 구분된다.^[STC-C008] (Mem0 2026a; Mem0 2026b) 이 구분은 derived memory 를 append-only lineage 로 남기고 삭제/충돌처리를 별도 operation 으로 관리할 수 있게 한다.

Letta 의 tagged sleeptime group 은 response 뒤 asynchronous agent 를 scheduling 해 durable memory 를 수정한다. ^[STC-C009] (Letta 2026b) 즉 main agent 와 memory agent 를 분리함으로써 current response latency 와 background memory-writing policy 를 decouple 한다.

Zep/Graphiti 는 episode, entity, relation 을 temporal version 으로 표현하며 per-user model-weight update 가 아니다.^[STC-C010] (Rasmussen et al. 2025; Zep 2026a; Zep 2026b) Temporal edge 와 supersession 은 mutable fact 를 weights 에 덮어쓰는 대신 history 와 current truth 를 함께 보존하는 강점을 가진다.

ReasoningBank 는 experience 에서 reusable reasoning strategy 를 추출해 external strategy memory 에 저장하며, broad online weight consolidation 이 아니다.^[STC-C012] (Ouyang et al. 2025; Google Research 2026) 이 계열은 “무엇을 기억할지” 와 “main model 을 바꿀지” 를 분리한다.

7.2 Company evidence 를 과대 해석하지 않기

Meta 의 personalized-agent 와 HyperAgents 공개 연구는 feedback-conditioned behavior, program memory, archive 를 강조하지만 fleet-scale per-user foundation-weight sleep 을 입증하지 않는다. ^[STC-C013] (Meta AI 2026b; Meta AI 2026a) “personalization 연구” 가 곧 “nightly user weight training product” 라는 inference 를 막기 위해 product, research prototype, benchmark 를 구분한다.

Microsoft STATE-Bench 는 state tracking, adaptation, efficiency 를 함께 평가하는 memory benchmark 이지, sleep-training product 공개가 아니다.^[STC-C014] (Microsoft 2026a; Microsoft 2026b) Benchmark 의 등장은 업계 문제의식을 보여 주지만 특정 mechanism adoption 의 증거는 아니다.

7.3 Parametric sleep: 직접적이지만 research-scale 인 증거

Language Models Need Sleep 은 model 이 self-modify 하고 replay-like training 으로 memory 를 consolidate 하는 parametric sleep 의 직접 evidence 다. 그러나 보고된 regime 은 research-scale 이

며 production deployment proof 는 아니다.^[STC-C015] (Behrouz et al. 2026a)

Parametric path 의 잠재 이점은 retrieval 없이 behavior/skill 에 반영되고, repeated query 에서 context token 과 external hop 을 줄이며, procedural pattern 을 distributed representation 에 통합할 수 있다는 것이다. 동시에 다음 조건을 해결해야 한다.

- multi-cycle old/new retention 과 capacity saturation,
- base model release 마다 adapter/state rebase,
- data lineage 와 targeted delete/unlearning,
- candidate validation, fleet rollout, rollback,
- user isolation 과 poisoned-memory defense,
- per-user job scheduling 과 serving artifact explosion.

7.4 Hybrid promotion

External record 를 authoritative evidence plane 으로 유지하고 parametric state 를 reconstructable serving cache 로 취급한다. Admission 은 안정성, reuse, scope, evidence, deletion risk 를 평가하고 promotion 은 adapter/compiled state 부터 시작한다. Global weights 는 여러 cohort 에 일반화되고 independent evidence 가충분할 때만 periodic refresh queue 로 보낸다.

Table 7.1: External, parametric, hybrid sleep 의 운영 성질

성질	External	Parametric	Hybrid
write/delete	item-level 쉬움	diffuse, 어려움	source 는 쉬움, cache re-build
provenance	explicit link	간접/불완전	external lineage 가 기준
wake cost	retrieval+context	낮을 수 있음	hot pattern 만 절감
capacity	storage/index 한도	interference/state 한도	tier 별 budget
portability	model-independent 가 능	base-dependent	external portable, delta conditional
rollback	record/index version	artifact/model rollback	source replay 로 재구성
best content	changing facts, episodes	repeated stable behavior	fact+behavior mixture

주의 “기억했다” 의 세 수준

정보가 parameter 에 encoding 되었다고 해서 query 에서 access 되거나 다른 fact 와 composition 되는 것은 아니다. 저장 (storage), 접근 (access), 활용 (composition) 을 separate metric 으로 보고, external/parametric 양쪽에 같은 task slice 를 적용한다.

Strongest-alternative matched comparison

이 장은 “아무 기억도 없는 fixed model” 이 아니라 문제별 strongest plausible alternative 와 STC 를 matched total work, state, quality, governance 에서 비교한다. Sleep phase 의 장점이 transform algorithm 자체의 장점인지 scheduling/validation 의 장점인지도 분리한다.

8.1 비교 원칙

1. 같은 base model, wake trace, validation set, hardware budget 을 사용한다.
2. external index build 와 parametric training 의 ingest-validation-publish 비용을 모두 포함한다.
3. per-query 비용은 같은 future query trace 에서 측정하고 one-time cost 와 분리한다.
4. capacity budget 을 맞추고 old/new/safety/delete slice 를 여러 sleep cycle 에 걸쳐 평가한다.
5. source-of-truth, rollback, base-version change 를 operational metric 으로 포함한다.

8.2 문제별 matched baseline

Table 8.1: 문제별 strongest non-STC baseline 과 조건부 verdict

Problem	Strongest baseline	STC candidate	Conditional verdict
one-off long document	long-context inference	compile/summarize before query	reuse 없으면 baseline 우세
repeated corpus QA	RAG + cache	background index/-summary/latent compile	valid query reuse 가 break-even 초과 시 STC 가능
mutable factual memory	temporal graph	scheduled conflict resolution	destination 은 external sleep 우세
stable user preference	profile retrieval	user adapter promotion	high reuse-low churn-easy rollback 에서 parametric 후보
rare correction	ROME/MEMIT or record edit	sleep batch edit/replay	immediate edit 우세, sleep 은 multi-edit integration
continual skill	replay+PEFT/global refresh	per-domain consolidation	bounded state 에서 retention 입증 시 후보
agent strategy	reflection/ReasoningBank	multi-episode strategy synthesis	external sleep evidence 가 강함

다음 쪽에 계속

Problem	Strongest baseline	STC candidate	Conditional verdict
serving efficiency	prefix/state cache	sleep state compaction	state reuse 와 compatibility 가 높을 때만
global knowledge	scheduled continued PT	wake-derived global sleep queue	이름 보다 cadence-data quality 비교
privacy-local adaptation	on-device file/RAG	pro-on-device LoRA/compaction	thermal/endurance 포함 matched test 필요

8.3 Hybrid recommendation

Hybrid: external system-of-record → selective reversible promotion

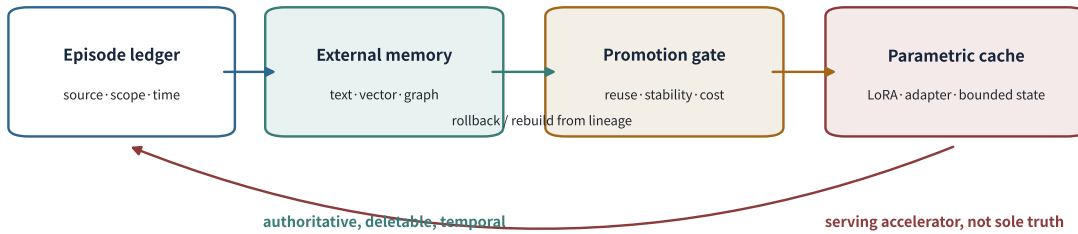


Figure 8.1: External system-of-record 와 selective reversible parametric promotion. External evidence plane 은 authoritative 하고, parametric state 는 break-even 을 통과한 serving cache 로 취급한다. [STC-F006]

출처/제작: Original synthesis; SRC-STC-0030, SRC-STC-0066 대체 설명: Episode ledger 가 external text-vector-graph memory 와 promotion gate 를 지나 LoRA-adapter cache 로 이어지고 rollback 은 provenance 를 따라 원점으로 돌아가는 구조.

Hybrid 는 항상 절충안이라서 선택하는 것이 아니다. 서로 다른 content class 가 요구하는 property 가 다르기 때문이다. Changing fact 와 rights-sensitive episode 는 external source-of-record 에 남고, repeated stable behavior 와 procedure 는 adapter 또는 bounded neural state 로 승격할 수 있다. 승격 뒤에도 original evidence 를 남겨 rebase, delete, audit 가 가능해야 한다.

Promotion score 의 예는 다음과 같다.

$$P(e) = \alpha Reuse(e) + \beta Stability(e) + \gamma WakeSaving(e) - \delta DeleteRisk(e) - \eta Interference(e) - \zeta RebaseCost(e). \quad (8.1)$$

이는 학습된 universal score 가 아니라 측정 항목을 빠뜨리지 않는 policy template 다. Threshold 는 user, tenant, device, content class 별로 다르며 uncertainty 가 크면 external 에 남긴다.

8.4 STC 가 이기지 못하는 영역

- 한 번만 읽는 document 와 즉시 답이 필요한 query,
- 빠르게 변하고 exact citation·history·delete 가 중요한 fact,
- future query distribution 이 예측 불가능한 low-reuse memory,
- base model churn 이 adapter lifetime 보다 짧은 service,
- sleep candidate 를 검증할 held-out trace 가 없는 초기 product,
- state migration 이 compute saving 보다 큰 disaggregated infrastructure.

핵심 가장 강한 현재 판단

External-memory sleep 은 retrieval 의 대체라기보다 writer·compactor·conflict resolver 를 background 로 옮기는 발전이다. Parametric sleep 은 external system 보다 본질적으로 더 긴 기억이 아니라, 반복되는 stable pattern 의 serving cost 를 선불로 지불하는 optimization 이다.

Evidence trajectory: 무엇이 실제로 성숙하고 있는가

이 장의 목적은 논문 수를 세어 momentum 을 선언하는 것이 아니라, 서로 다른 종류의 증거가 어떤 주장을 허용하는지 분리하는 것이다. 연구용 알고리즘의 controlled experiment, 공개 code, 공식 product documentation, production trace, independent incident report 는 같은 무게가 아니다. 또한 “sleep” 이라는 단어가 없는 continual learning, external memory, state caching 연구도 lifecycle 의 일부를 직접 다룰 수 있고, 반대로 생물학적 은유만 사용하는 연구는 operational STC 의 증거가 아닐 수 있다.

9.1 Date-frozen corpus 와 재현 가능한 경계

본 연구는 2026-08-05 를 source freeze 로 고정했다. Registry 에는 paper, official release, documentation, tagged repository 를 합쳐 73 개의 primary/official source 가 들어 있다. [STC-C001] 각 source 는 stable ID, canonical URL, 공개일, 접근일, source type 을 가지며, 본문의 claim 은 prose citation 이 아니라 마지막 ledger 의 exact locator 로 다시 연결된다. 이 수는 검색 결과의 개수가 아니라 중복 제거와 source-kind 판정을 마친 audit surface 다.

동결 시점에 명시적 background memory synthesis 를 설명한 공개 production lineage 는 적어도 OpenAI Dreaming 과 Mem0 Dream 두 개였다. [STC-C002] (OpenAI 2026; Mem0 2026a) Letta 의 tagged sleeptime code 와 Zep/Graphiti 의 temporal provenance 는 같은 lifecycle 을 다른 component 에서 보강하지만, product-scale comparative effectiveness 를 대신하지 않는다. 공식 release 는 “기능이 존재한다” 를 지지할 수 있어도 “strongest baseline 보다 우월하다” 를 지지하지 못한다.

검색은 saturation 을 과장하지 않는다. 12 개 cluster 중 2 개만 bounded closure, 5 개는 active frontier, 5 개는 measurement gap 으로 닫혔다. [STC-C003] 특히 utility-sleep compute, utility-memory budget, retention-update count, bytes moved-placement curve 가 없는 cluster 는 source 가 많아도 “saturated” 로 부르지 않았다. 이 규칙은 2026-08-05 이후의 발견이 결론을 바꿀 수 있음을 문서 구조에 남긴다.

9.2 Evidence maturity 와 expected value 는 다른 축이다

Expected value 가 높아도 evidence 가 E1 이면 high-upside research bet 이다. 반대로 E5 인 query-time retrieval 이 특정 high-reuse procedural workload 에서 가장 값싼 해답이라는 뜻도 아니다. 본 논문은 problem severity, quality advantage, cost advantage, evidence maturity, deployment fit, governance, industry momentum, academic momentum 을 각각 기록하고 평균 점수 하나로 숨기지 않는다.

9.3 Industry 와 academia 의 서로 다른 수렴

Industry 에서 가장 분명한 경로는 episode 보존 → contradiction/dedup → scheduled pattern synthesis → future retrieval → user control/provenance 다. Academia 의 경로는 더 넓다. Replay 와 continual-learning regularization, test-time neural memory, memory writer/router RL, rate-distortion compaction, self-evolving strategy memory, sequential fact reachability 가 서로 다

Table 9.1: 본문에서 사용하는 evidence maturity ladder

Level	최소 근거	허용되는 문장	허용되지 않는 확대 해석
E0	아이디어 · 명칭	제안되었다	작동한다
E1	단일 synthetic/offline 실험	제한된 setting 에서 관측됐다	일반화된다는
E2	다중 task/model 또는 code	조건부로 지지된다	production-ready 다
E3	strong baseline, ablation, long horizon	평가 범위에서 경쟁력 있다	fleet economics 가 성립한다
E4	공식 production lifecycle	공개 제품에서 동작한다	scientific optimum 이다
E5	independent deployment trace-incident	운영 성질이 독립 확인됐다	다른 workload 에도 보편적이다

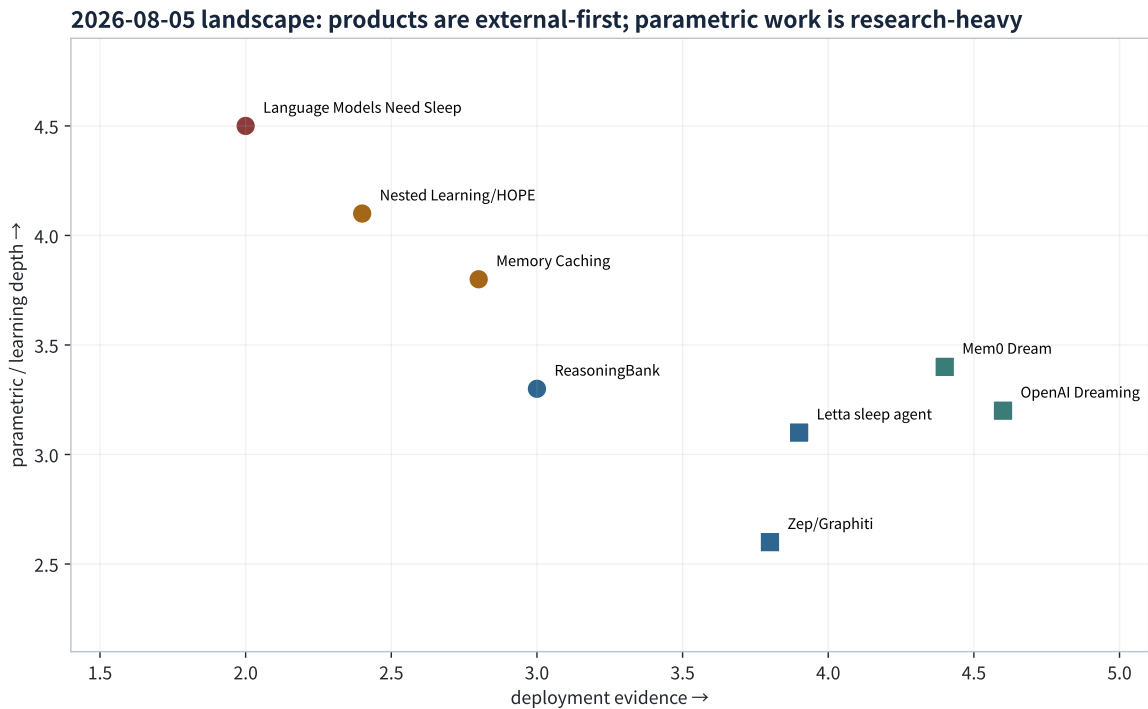


Figure 9.1: Source freeze 시점의 industry-academia landscape. Industry 는 external synthesis 와 temporal provenance 에, academia 는 continual learning, neural memory, learned writer 와 capacity theory 에 더 넓게 분포한다. 좌표는 corpus 의 정성적 audit 를 시각화한 것이며 시장 규모가 아니다.
[STC-F014]

출처/제작: Original synthesis; SRC-STC-0054, SRC-STC-0056, SRC-STC-0057 대체 설명: 제품과 연구 lineages 를 external lifecycle, learned memory policy, parametric consolidation, benchmark/theory 축에 배치한 지도.

른 이름으로 같은 admission-transform-destination-validation 문제를 만든다. (Behrouz et al. 2025; Behrouz et al. 2026a; Colaco and Lahjouji 2026; Ouyang et al. 2025; Unknown organization 2026a)

따라서 momentum 에 대한 정확한 표현은 “external background transformation 은 I3/E4 에 도달했고, learned external policy 는 빠른 E2-E3 frontier 이며, broad per-user parametric sleep 은 E1-E2” 다. 이 세 문장을 하나의 “STC 가 뜬다” 로 합치면 투자 방향과 실험 설계가 모두 왜곡된다.

9.4 번역 corpus 가 담당하는 역할

전체 source 를 모두 번역하는 대신, explicit sleep, biological/computational precursor, continual learning, external memory, neural memory, capacity, negative evidence 를 가로지르는 15 편을 balanced translation corpus 로 고정했다. [STC-C004] 번역본은 의견을 추가하는 secondary review 가 아니라 원문의 section-figure-limitation 을 보존하는 독해 보조물이다. 반면 이 Study 는 그 source 들을 problem 과 lifecycle 기준으로 재조합하는 분석이다. 두 산출물을 분리해야 번역자의 해석이 원저자의 주장처럼 보이지 않는다.

핵심 Evidence trajectory 의 결론

새로운 축의 가장 강한 증거는 하나의 breakthrough paper 가 아니다. Continual-learning mechanism, external-memory product, asynchronous implementation, temporal provenance, realistic benchmark 가 서로다른 층에서 같은 lifecycle boundary 를 채우기 시작했다는 점이다. 다만 parametric destination 의 장기 retention·delete·rollback·fleet cost 를 동시에 보여 주는 공개 증거는 아직 비어 있다.

Promisingness: 새로운 축은 정말 최선인가

질문은 “sleep 이라는 이름이 흥미로운가” 가 아니라, 동일한 wake evidence 와 future query 를 주었을 때 deferred transformation 이 long context, RAG, periodic refresh, model editing, wake TTT 보다 더 나은 lifetime frontier 를 만드는가다. 여기서 frontier 는 quality 만이 아니라 total retained state, later-wake cost, freshness, delete, rollback, wake-SLA 까지 포함한다.

10.1 현재 증거가 지지하는 범위

공개 industry evidence 는 external-memory sleep 과 background synthesis 에서 가장 강하며, 광범위한 per-user foundation-weight sleep 에서는 그렇지 않다.^[STC-C038] (OpenAI 2026; Letta 2026b; Mem0 2026a; Zep 2026a) 이는 external 방식이 영원히 우월하다는 뜻이 아니라, 이미 배포된 lifecycle 과 아직 연구-scale 인 parametric mechanism 을 구분한다는 뜻이다.

Broad parametric sleep 은 continual retention, rollback, deletion, security, fleet serving economics 를 동시에 입증하지 못했다.^[STC-C039] “Language Models Need Sleep” 은 self-modification 과 consolidation 의 direct evidence 지만 연구 규모의 method result 이고, sequential fact learning 은 weight 에남은 정보가 normal behavior 에서 접근 불가능해질 수 있음을 보여 준다. External memory 도 poisonable 하기때문에 단순히 외부에 두면 문제가 끝나는 것은 아니다. (Behrouz et al. 2026a; O’Neill 2026; Chen et al. 2024b)

가장 신뢰할 중기 구조는 auditable external system-of-record 에 원 증거를 남기고, stable 하고 high-reuse 인 일부만 reversible adapter 또는 bounded parametric state 로 승격하는 hybrid 다.^[STC-C040] (Chen et al. 2024a; Hu et al. 2021; OpenAI 2026; Zep 2026b) Parametric artifact 는 기억의 원장이 아니라 serving 을 위한 compiled cache 로 취급한다. Base model, consent, evidence validity 가 바뀌면 external source 에서 다시 만들 수 있어야 한다.

10.2 여섯 개의 핵심 verdict

Table 10.1: 핵심 주장별 조건부 verdict 와 반증 조건

Claim	Verdict	근거가 허용하는 범위	Confidence 를 낮출 관측
새 scaling phase 인가	functional level 에서 yes	release-time training 도 current-query reasoning 도 아닌 durable later-wake state budget	matched synchronous maintenance 와 차이가 전혀 없음
mainstream 인가	lifecycle 은 likely, 명칭 parametric form 은 uncertain	persistent agent 의 background synthesis/validation	주요 platform 이 scheduled transform 을 채택하지 않음
long context 의 최선인가	generally no	predictable high-reuse query 에서 compile break-even 가능	high reuse 에서도 quality/cost crossover 없음

Claim	Verdict	근거가 허용하는 범위	Confidence 를 낮출 관측
continual learning 해결인가	useful phase, solution theorem 은 아님	replay·regularization·isolation 을 실행할 시간 경계 제공	repeated cycle 에서 빠른 capacity knee
parametric 이 external 보다 우월한가	content-dependent	stable skill 은 compile, volatile exact fact 는 external	단일 medium 이 모든 축에서 지배
device opportunity 인가	workload 는 yes, 제품 claim 은 별도 검토	version, lineage, compaction, mixed read/write 가 새 trace 생성	실제 trace 가 작은 metadata job 에 머뭇

10.3 문제별 strongest alternative 와 sleep 의 조건

One-off long context. 한 번 읽고 답하는 문서는 FlashAttention 기반 long context 또는 RAG 가 정확한 evidence 와 citation 을 보존한다. Sleep compilation 은 query 전에 비용을 내고 정보 손실과 staleness 를 추가하므로 reuse 예측이 없으면 열세다.

Mutable factual memory. Temporal graph 와 versioned retrieval 이 현재값, 과거값, supersession, delete 를 직접 표현한다. Parametric write 가 경쟁하려면 normal-query reachability 와 exact reversal 을 추가로 증명해야 한다.

Repeated procedure 와 strategy. 동일 repository, tool, enterprise procedure 가 반복되면 retrieval/context token 과 reasoning latency 를 매번 지불한다. 이때 background synthesis, learned strategy memory, adapter compilation 이 유망하다. 다만 exact-repeat 가 아니라 held-out semantic transfer 로효과를 측정해야 한다.(Ouyang et al. 2025; Google Research 2026)

Global common knowledge. 사용자별 sleep 보다 centralized continued pre-training 과 periodic refresh 가 더 넓게 amortize 된다. STC 가 이기려면 central release cadence 보다 빠른 local adaptation 또는 공유할 수 없는 private evidence 라는 조건이 필요하다.

On-device personalization. Idle window, privacy, intermittent connectivity 는 sleep scheduling 과 잘 맞지만 thermal, battery, write endurance, recovery state 까지 포함한 측정이 없다면 가능성만 말할 수 있다. Cloud 비용을 device 비용으로 옮긴 것을 효율 향상으로 세면 안 된다.

10.4 Workload 별 판단

10.5 의사결정 규칙

새 evidence e 를 destination j 로 compile 하는 decision 은 단순 score 가 아니라 최소 다목적 제약이다.

$$\text{promote}(e, j) \iff \begin{cases} \hat{R}_v(e) \Delta U_j(e) > C_{life}(e, j), \\ P(\text{valid at reuse}) \geq \tau_v, \\ \text{delete/rollback/compatibility gates pass,} \\ \Delta U_{old}, \Delta U_{safety} \geq -\epsilon. \end{cases} \quad (10.1)$$

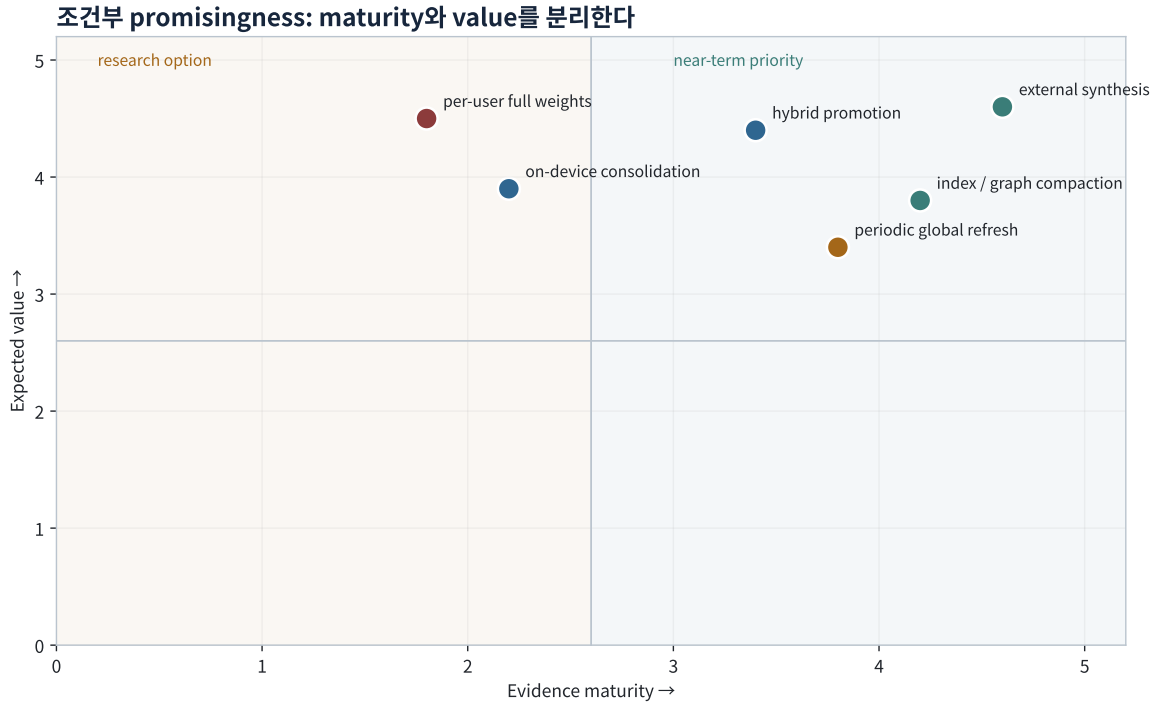


Figure 10.1: Evidence maturity 와 expected value 를 분리한 conditional verdict. 오른쪽 위로 갈수록 현재 증거와 잠재 가치가 함께 높다. Parametric sleep 은 잠재 가치가 커도 성숙도가 낮은 연구 bet 로 남는다. [STC-F005]

출처/제작: Original synthesis; SRC-STC-0052, SRC-STC-0063, SRC-STC-0021 대체 설명: External synthesis, learned writer, adapter promotion, broad weight update 를 성숙도와 기대 가치의 두 축에 배치한 산점도.

Table 10.2: Workload 별 expected value 와 현재 evidence maturity

Workload	가치	성숙도	현재 추천
multi-year personal assistant	높음	E4 external / E1 parametric	external synthesis 우선
enterprise procedure/coding	매우 높음	E2-E3	external strategy + selective adapter 실험
one-off document QA	낮음 - 중간	E5 baseline	long context/RAG 우선
recurring technical corpus	높음	E2	reuse sweep 뒤 compile
universal current facts	중간	E5 refresh/RAG	global refresh + retrieval
private local adaptation	높음	E1	thermal/endurance 포함 pilot
high-churn profile	external 높음	E4 external	versioned external only
robotics/autonomy	높음	E1-E2	safety-gated domain sleep

하나라도 모르면 external 에 남기는 것이 기본값이다. 이 규칙은 STC 를 특별 대우하지 않는다. External summary 도 원본을 버리거나 false merge 를 만들 수 있으므로 같은 validation 과 lineage gate 를 통과해야한다.

핵심 Promisingness verdict

Sleep-time compute 은 하나의 universal method 가 아니라 미래 wake 를 위해 비용을 선불 지불하는 lifecycle class 다. 현재 가장 promising 한 부분은 external background transformation 과 learned memory policy 이고, 가장 high-upside 지만 불확실한 부분은 selective reversible parametric promotion 이다. Continuous per-user base-weight update 는 현재 strongest default 가 아니다.

Mainstream scenarios: lifecycle 과 이름의 미래

미래를 하나의 확률 숫자로 압축하면 workload 차이를 잃는다. Consumer assistant 는 external-memory dominant 이면서, enterprise coding agent 는 hybrid promotion 을 쓰고, foundation-model vendor 는 periodic global refresh 를 유지할 수 있다. 따라서 다섯 시나리오의 상호 배타적 시장 예측이 아니라 기술·경제 조건에 따른 system shape 다.

더 중요한 언어적 주의가 있다. 이 lifecycle 이 주류가 되어도 “sleep-time compute” 이라는 명칭 이 승리할 필요는 없다. 공개 system 은 dreaming, reflection, synthesis, memory, self-evolution, consolidation 이라는 서로 다른 말을 쓴다.^[STC-C042] (OpenAI 2026; Google Research 2026; Letta 2026a; Mem0 2026a) 따라서 adoption 은 keyword count 가 아니라 deferred execution, wake-derived input, durable destination, later-wake reuse 라는 operational signature 로 추적해야 한다.

11.1 다섯 개의 조건부 시나리오

Table 11.1: 시나리오별 trigger, leading indicator, falsifier

Scenario	Trigger	Leading indicators	Falsifiers
External dominant	provenance/delete 요구, frequent base upgrade, retrieval 개선	pattern/supersede/cadence API, portable temporal memory, memory incident tooling	procedural skill 을 지속적으로 잃거나 retrieval cost 가 고정 병목
Hybrid promotion	calibrated reuse predictor, cheap adapter validation/routing, source lineage	promote/demote/rebuild API, adapter catalog, reuse sweep benchmark	dual-state consistency 와 version churn 이 saving 을 상쇄
Periodic refresh	global amortization 우세, local history retention 제한	release cadence 단축, central synthetic/eval factory, source-based adapter regeneration	local procedure 가 release 사이에 빠르게 변함
STC niche	높은 reuse 가 일부 domain 에만 존재, validation cost 큼	domain/cohort adapter, ROI/pressure trigger, repeated-task gain	general assistant 에 서도 background synthesis 가 표준화
STC broad	hundreds-cycle 안 정성, bounded delete/rollback, stable queue, migration economics	wake/sleep quota 와 SLO, durable state handle, lifecycle scaling curves	validation-state fan-out·privacy cost 가 benefit 보다 빠르게 증가

11.2 현재 base case

동결된 증거와 가장 일치하는 조합은 세 줄이다.

1. **Near term: external-memory dominant.** Background synthesis, dedup, conflict resolution, temporal graph, provenance 가 foundation weight 를 바꾸지 않고 먼저 확산한다.
2. **Evolution: high-reuse stable skill 에서 hybrid promotion.** External source-of-record 를 버리지 않고 adapter, expert, latent cache 를 제한적으로 compile 한다.
3. **Coexistence: global common knowledge 는 periodic refresh.** 모든 durable learning 을 사용자별 sleep job 으로 분해하지 않는다.

STC niche 의 가장 강한 형태, 즉 general assistant 에는 background transformation 이 거의 없을 것이라는 예측은 OpenAI 와 Mem0 evidence 로 이미 약해졌다. 반대로 continuous per-user parametric learning 이 표준이 될 것이라는 broad scenario 는 repeated-cycle E3 와 production E4 가 없다.

11.3 분기점을 미리 보여 주는 signal

Table 11.2: Quarterly signal dashboard

관측할 signal	의미	어느 방향을 강화하는가
public per-user weight/adapter update 와 rollback trace	parametric lifecycle 성숙도	hybrid/broad
memory provenance-supersede-delete API 확장	external governance maturity	external/hybrid
reuse predictor calibration 과 top- k yield	compile economics	hybrid/niche
base upgrade 후 adapter/index rebuild cost	artifact lifetime	refresh 대 hybrid
100–1,000 cycle retention/plasticity curve	lifelong claim 가능성	broad 또는 반증
background job bytes, energy, queue age	실제 infra workload	모든 시나리오의 device 규모
learned writer/router 의 cross-domain result	main-model update 대안	external/hybrid
on-device idle training trace	privacy-local fit	niche/broad

11.4 시나리오에 강건한 연구 전략

예측에 베팅하기보다 공통 분모를 먼저 만든다. Immutable wake event 와 lineage, versioned candidate, validation attestation, atomic publish/rollback, total-state accounting 은 external-only 에서도 필요하고 hybrid 와 broad adoption 에서는 더 중요하다. Algorithm research 는

reuse·volatility·capacity sweep 으로 어떤 item 을 external 에 남기고 무엇을 promote 할지 학습한다. Hardware research 는 특정 “sleep accelerator” 명칭보다 state locality, mixed rewrite, snapshot, metadata tail latency 를 측정한다.

주의 Scenario 를 달력 예측으로 읽지 않기

이 장은 2028 년 market share 를 예측하지 않는다. 각 시나리오가 성립하려면 어떤 공개 evidence 가 나타나야 하고 무엇이 confidence 를 낮추는지 미리 고정한다. 새 정보가 들어오면 narrative 가 아니라 trigger 와 falsifier 를 갱신한다.

Scaling law: 법칙이 아니라 측정 가능한 가설부터

Pre-training scaling 은 통제된 data, parameter, compute 와 loss 의 관계를 다룬다. STC 에는 previous state, wake evidence, admission policy, memory medium, cycle count, future query reuse, validation 과 publication 이 추가되어 path-dependent 하다. 따라서 “accuracy versus sleep FLOPs” 한 곡선은 state 가증가했는지, data 가 좋아졌는지, future reuse 가 달랐는지 구분하지 못한다.

동결 corpus 의 어느 source 도 보편적인 sleep-time scaling law 를 확립하지 않는다.^[STC-C041] (Morris et al. 2025; Colaco and Lahjouji 2026; Unknown organization 2026b) 아래 식은 MEASURED accounting identity, ANALYTICAL break-even, FIT CANDIDATE, HYPOTHESIS 의 네 등급으로 분리한다. 관측 전 가설을 “law” 라고 부르지 않는 것이 이 장의 가장 중요한 결과다.

수식의 지위를 섞지 않는다



universal ‘more sleep FLOPs → better memory’ law: not established

Figure 12.1: Scaling statement 의 네 등급. Identity 는 장부를 닫고, break-even 은 제한된 가정 아래경계를 계산하며, fit candidate 는 sweep 에 맞추고, hypothesis 는 반증 실험을 기다린다.

^[STC-F007]

출처/제작: Original synthesis; SRC-STC-0035, SRC-STC-0050 **대체 설명:** 측정식에서 보편 법칙으로 갈수록 필요한 가정과 검증 범위가 증가하는 네 단계 사다리.

12.1 State 와 lifecycle work 를 먼저 닫는 identity

Answer-bearing retained state 를 destination 하나로 축소하면 안 된다.

$$B_{\text{retained}} = B_r + B_e + B_l + B_p + B_a + B_v, \quad (12.1)$$

여기서 B_r 은 canonical/raw evidence, B_e 는 text/vector/graph/index, B_l 은 latent·KV·recurrent state, B_p 는 adapter/expert/weight delta, B_a 는 optimizer·importance·router·teacher support, B_v 는 lineage·predecessor·rollback state 다. Active summary bytes 만 보고 “압축률” 을 계산하면 archive 와 recovery copy 가 사라진 것처럼 보이는 오류가 생긴다.

동시에 존재하는 candidate, pinned predecessor, migration, workspace, replica 까지 더한 peak 는

$$B_{peak,state}(t) = B_{retained}(t) + B_{candidate}(t) + B_{pinned}(t) + B_{migration}(t) + B_{workspace}(t) + B_{replica}(t), \quad (12.2)$$

$$B_{peak,physical}(t) = B_{base,resident}(t) + B_{peak,state}(t) + B_{execution}(t). \quad (12.3)$$

Physical maximum 은 각 항의 서로 다른 시간대 high-water mark 를 더하지 않고 같은 timestamp 에서평가한다. Separate wake/sleep fleet 가 contention 을 줄이면서 base model replica 를 늘릴 수 있으므로 incremental state 와 physical placement 를 둘 다 보고해야 한다.

Full-history replay 는 특히 위험하다. Cycle t 에서 이전 task 의 m_i 개를 모두 replay 하면

$$W_T = \sum_{t=1}^T \sum_{i=1}^t m_i = \sum_{i=1}^T m_i (T - i + 1). \quad (12.4)$$

$m_i = m > 0$ 이면 $W_T = mT(T + 1)/2 = \Theta(T^2)$ 다. Admission fraction 을 일정 비율 줄이는 것만으로는계수만 줄고 차수가 바뀌지 않는다. Bounded memory 를 주장하려면 expire, compaction, coreset, generated replay 또는 selective revisit 가 실제로 lifetime work 와 retained state 를 제한하는지 보여야 한다.

12.2 Valid reuse 가 만드는 compile break-even

Sleep artifact i 의 가치에서 raw memory count 나 sleep FLOPs 보다 유용한 후보 변수는 valid reusable evidence per lifecycle cost 다. [STC-C043] (Colaco and Lahjouji 2026; Ouyang et al. 2025; Unknown organization 2026a) 그 이유는 compile 이 한 번 비싸도 미래에 반복 사용되면 이득이고, 수천 개 memory 를 만들었어도 다시쓰이지 않으면 이득이 0 이기 때문이다.

Stationary 하고 quality 가 같다는 제한된 가정에서 reuse break-even 은

$$n^* = \frac{C_{compile} + C_{maint}}{\Delta C_{use}}. \quad (12.5)$$

Validity survival 과 discount 를 넣으면 horizon H 의 expected valid reuse 는

$$R_i(H) = \int_0^H \lambda_i(t) S_i(t) e^{-\omega t} dt. \quad (12.6)$$

Promotion 은 $R_i(H) \Delta U_{ij}$ 가 compile, serve, governance, risk cost 의 합보다 클 때만 후보가 된다. Future reuse 를 oracle 로 쓰지 않기 위해 실제 router 는 past-only feature 로 $\hat{R}_i$ 를 예측하고, calibration error 도 lifecycle cost 에 넣는다.

측정용 efficiency 는 scalar 를 강제하지 않고 먼저 Pareto vector 로 보고한다.

$$\eta_i = \frac{\Delta U_{later,i} R_{valid,i}}{C_{life,i}}, \quad (12.7)$$

$$C_{life} = C_{ingest} + C_{transform} + C_{train} + C_{validate} + C_{publish} + C_{serve} + C_{maint} + C_{delete/rollback}. \quad (12.8)$$

Dollar, joule, byte, second 를 임의 환율로 합치지 않는다. Utility floor 를 통과한 뒤 비용을 비교하거나, cost cap 아래 utility 를 비교하는 방식이 더 재현 가능하다.

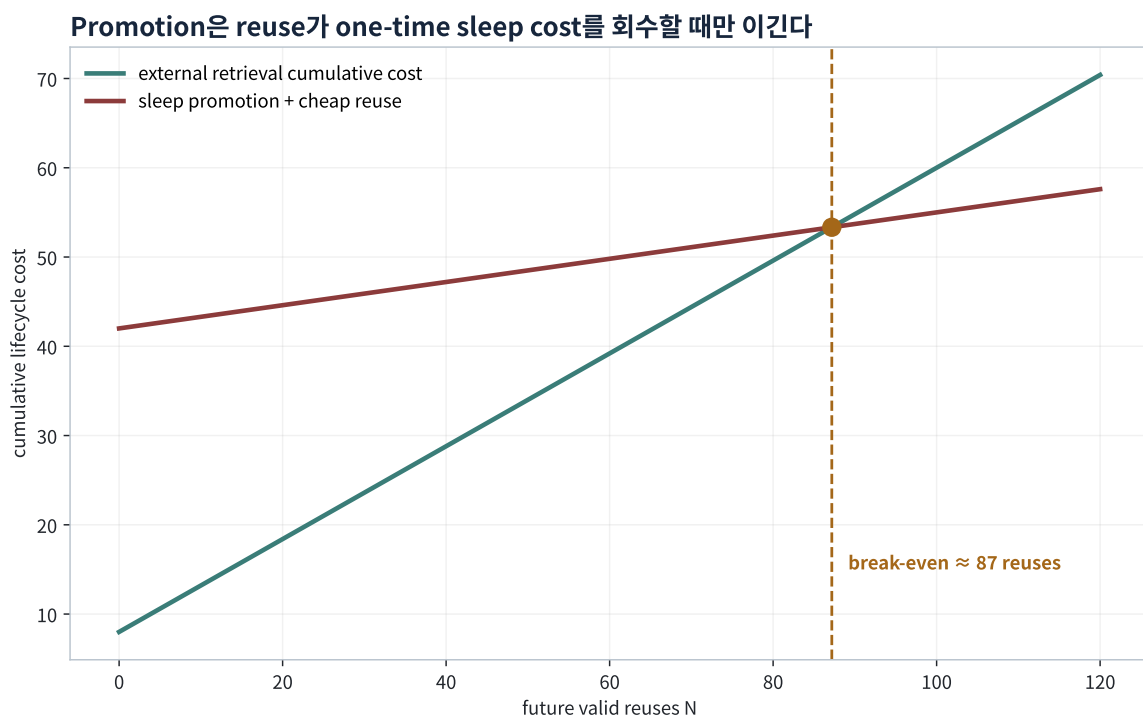


Figure 12.2: One-time sleep cost 와 future valid reuse 의 analytical crossover. Break-even 왼쪽은 external retrieval 이, 오른쪽은 compiled artifact 가 유리할 수 있다. Quality-governance 차이가 있으면 경계선이 이동한다.

[STC-F008]

출처/제작: Original analytical redraw; SRC-STC-0027, SRC-STC-0052 대체 설명: 초기 compile 비용을 수평선으로, reuse 에 따라 누적되는 per-use saving 을 상승선으로 나타낸 교차 그래프.

12.3 Finite state가 만드는 capacity knee

고정된 finite state 는 positive-entropy stream 을 같은 distortion 으로 영원히 보존할 수 없다. Mutable state 가 $b_{mutable}$ bits 이고 conditionally independent answer family 의 rate-distortion cost 가 $R_{Y_i|Q_i, \Theta_0}(D_i)$ 라면 제한된 조건 아래

$$\sum_{i=1}^n R_{Y_i|Q_i, \Theta_0}(D_i) \leq H(M_n | \Theta_0) \leq b_{mutable}. \quad (12.9)$$

이는 facts-per-parameter 보편 상수가 아니다. 하지만 계속 들어오는 novel evidence 에 대해 state growth, distortion 증가, forgetting, external offload 중 적어도 하나가 필요하다는 점은 분명히 한다.

Finite memory 에서는 marginal consolidation utility 가 떨어지고 interference, compaction 또는 delete cost 가 급증하는 capacity knee 가 나타날 것으로 예상된다.^[STC-C045] (Morris et al. 2025; Colaco and Lahjouji 2026; O'Neill 2026) Physical storage knee, normal-query reachability knee, retrieval index knee, new-task plasticity knee, governance knee, hot-state residency knee 는 서로 다른 위치에 있을 수 있다. 따라서 “parameter 가 아직 남았다” 는 말은 behavioral capacity 의 반증 이 아니다.

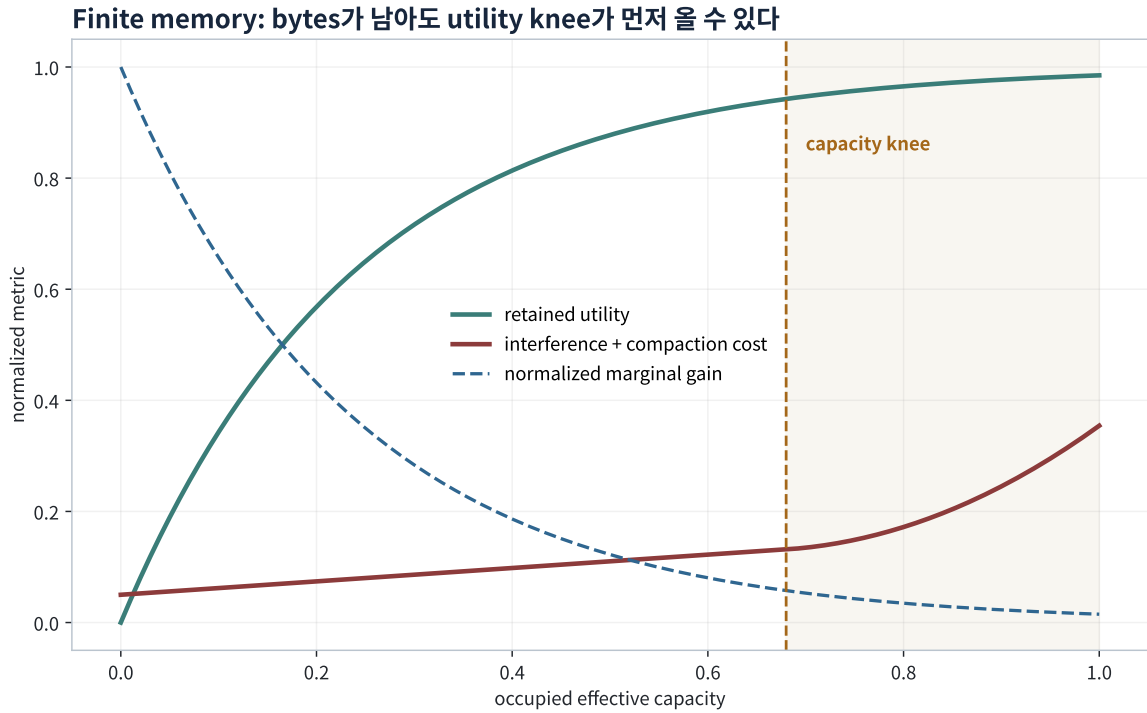


Figure 12.3: Finite-memory capacity knee 의 가설적 형태. Physical occupancy 가 한계에 닿기 전에 behavioral reachability 또는 plasticity 가 임계값 아래로 내려갈 수 있다. 곡선은 설명용이며 측정 결과가 아니다.

[STC-F009]

출처/제작: Original hypothesis redraw; SRC-STC-0034, SRC-STC-0036 대체 설명: Evidence load 가 증가할 때 retention 과 plasticity 가 서로 다른 knee 에서 감소하는 두 곡선.

Controlled sweep 에서는 retention threshold τ_{ret} 을 먼저 고정하고

$$\text{logit } p_{ret}(N_a) = \text{logit } \tau_{ret} - \kappa \log(N_a / K_{eff}) \quad (12.10)$$

같은 segmented knee candidate 와 generalized mean, min-bottleneck, monotone nonparametric competitor 를함께 맞춘다. 전 구간 power law 하나를 강제하지 않는다.

12.4 Cadence 와 queue 의 내부 optimum

Optimal cadence 는 staleness, batching efficiency, interference 와 wake-SLA impact 를 균형시켜야 한다. Continuous update 나 무한히 긴 batch 어느 쪽도 일반적으로 최적이지 않다.^[STC-C044] (Behrouz et al. 2025; Behrouz et al. 2026b; Mem0 2026a)

Run cost C_s , interval τ , smooth staleness penalty $a\tau^p$ 라는 제한에서

$$\bar{C}(\tau) = \frac{C_s}{\tau} + a\tau^p, \quad \tau^* = \left(\frac{C_s}{ap} \right)^{1/(p+1)}. \quad (12.11)$$

Event rate r_e , fixed job cost F , linear delay harm h 이면 $\tau^* = \sqrt{2F/(hr_e)}$ 다. Burst arrival, hard deadline, multi-resource blocking, capacity-triggered regime change 에서는 이 closed form 이 깨진다. 이때 queue recursion $Q_{t+1} = \max\{0, Q_t + A_t - S_t\}$ 와 P99 job age 를 직접 시뮬레이션해야 한다.

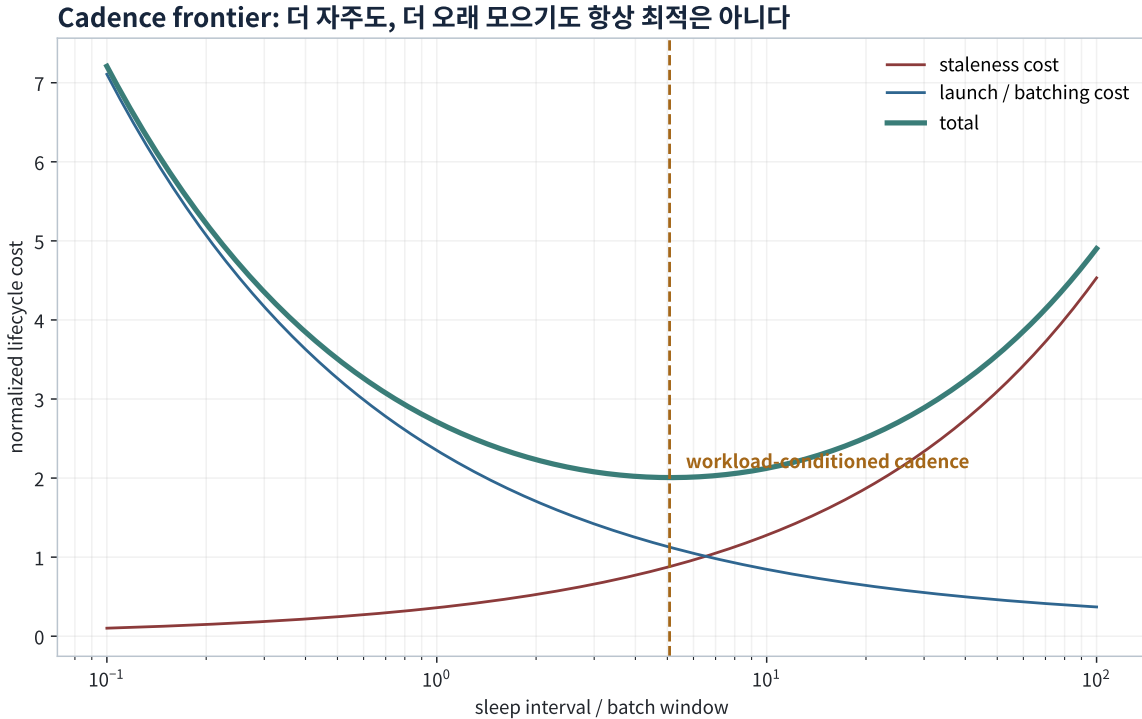


Figure 12.4: Cadence frontier. 너무 잦은 sleep 은 fixed overhead 와 interference 가 크고, 너무 늦은 sleep 은 staleness 와 queue age 가 크다. 내부 optimum 의 위치는 workload 와 resource vector 에 따라 변한다.
[STC-F010]

출처/제작: Original analytical redraw; SRC-STC-0020, SRC-STC-0063 대체 설명: 짧은 interval 의 batching 비용과 긴 interval 의 staleness 비용이 합쳐져 U 자 total cost 를 만드는 그래프.

12.5 Fit candidate 와 법칙 승격 조건

첫 fit candidate 는 saturating return 에 interference, staleness, risk penalty 를 분리한다.

$$\Delta U = A \left[1 - \exp \left(-k F_s^\alpha E_{eff}^\beta R_v^\gamma \right) \right] - \Phi_{int} - \Phi_{stale} - \Phi_{risk}. \quad (12.12)$$

E_{eff} 는 authorization, confidence, diversity 를 통과한 canonical evidence 이고 R_v 는 valid future reuse 다. Competitor 는 log-linear, floor 가 있는 power law, segmented knee, generalized mean, nonparametric monotone surface 다. Sign change 나 knee 가 보이면 global power law 를 기각한다.

진짜 STC scaling law 라 부르려면 (1) unit 과 state boundary 를 고정하고, (2) model size·workload·medium·hardware 를 둘 이상 포함하며, (3) strongest alternative 와 total lifecycle cost 를 맞추고, (4) held-out condition 을 예측하고, (5) knee 를 숨기지 않으며, (6) contamination·recursive depth·capacity·cycle control 을 통과하고, (7) 실패한 형태를 공개해야 한다. 그 전까지의 정직한 산출물은 identity, restricted break-even, workload-conditioned empirical surface 와 falsifiable hypothesis 다.

System infrastructure: wake, sleep, memory 를 어떻게 잇는가

세 개의 logical plane 이 곧 세 개의 physical cluster 를 뜻하지 않는다. 필요한 것은 latency-critical wake execution, throughput-oriented candidate construction, transactional publication 의 책임 분리다. Load 와 state locality 가 허용하면 같은 accelerator pool 을 time-partition 할 수 있고, contention 또는 privacy 가 크면 분리할 수 있다. 배치는 brain analogy 가 아니라 bytes, queue, SLO 측정으로 결정한다.

13.1 세 plane 과 하나의 publication transaction

1. **Wake plane:** inference, retrieval, optional fast learning, event append 를 latency SLO 안에서 수행하고, 하나의 authorized serving head 를 pin 한다.
2. **Sleep plane:** immutable wake snapshot 을 읽어 extract, compact, graph update, replay, distill, adapter tune, policy update, rebuild, verify job 을 수행한다. 결과는 candidate 일 뿐 live state 가 아니다.
3. **Memory/control plane:** raw evidence 와 derived views 의 lineage, version, authorization, capacity, atomic publish, rollback 을 관리한다.

Sleep transaction 은 다음처럼 표현할 수 있다.

$$T(X_v, M_v, B, P) \rightarrow \widehat{M}_{v+1} \xrightarrow[\text{compare-and-swap}]{\text{validate}} M_{v+1}. \quad (13.1)$$

X_v 는 immutable event range, M_v 는 published manifest, B 는 resource budget, P 는 frozen policy 다. Candidate 는 old/new/safety/delete/poison/system canary 를 통과한 뒤에만 manifest-last 로 publish 된다. Reader 는 요청 중 한 version 을 보며 partial vector/metadata/adapter state 를 섞어 보지 않는다.

13.2 Source-of-record 와 heterogeneous materialized views

Derived memory 의 governance basis 로는 updated weights 하나보다 external source-of-record 와 provenance graph 가 강하다.^[STC-C046](OpenAI 2025; Mem0 2026a; Zep 2026b) Raw event 도 retention policy 가 허용하는 동안만 canonical 이다. Text summary, vector, graph, latent/KV, adapter, shared weight 는 모두 source lineage 를 가진 materialized view 로 취급한다.

삭제는 tombstone 한 번이 아니다. Immediate access deny, online artifact invalidation, derived lineage rebuild, backup/key expiry, behavioral residual test 를 서로 다른 deadline 으로 기록한다. Old checkpoint 전체를 되살리는 rollback 은 그 사이의 delete/consent epoch 를 되돌릴 수 있으므로, authorized sub-root 만 새 generation 으로 재조합해야 한다.

Table 13.1: Memory medium 별 serving path 와 rollback contract

Medium	Wake path	Update/validation	Rollback basis
raw episode	selective retrieval	append, redact, expire	event/version log
semantic text	prompt injection	rewrite, faithfulness	source span + predecessor
vector/graph	ANN/traversal	re-embed, edge validity	index generation + episode
latent/recurrent	accelerator state read	compatibility, semantic probe	checkpoint or source rebuild
adapter/expert	model forward/router	train, merge, old/new canary	base digest + delta version
shared weight	model forward	full regression/unlearning	checkpoint + training lineage

13.3 Scheduler: operator, destination, cadence 를 분리한다

하나의 evidence region 에 대한 action space 는 NO_OP, RETAIN_EXACT, EXPIRE, DEDUP, MERGE, ABSTRACT, GRAPH_UPDATE, LATENT_COMPILE, ADAPTER_COMPILE, PARAMETRIC_PROMOTE, DEMOTE, REBUILD, QUARANTINE 를 포함한다. Learned policy 가 action 을 제안해도 privacy, tenant boundary, capacity, deadline, promotion gate 는 hard constraint 로 남긴다.

Batching key 는 base model/tokenizer 와 adapter shape, embedding/index version, privacy zone, deadline, operator, source shard 다. 같은 batch 가 효율적이라는 이유로 tenant security domain 을 합치지 않는다. Preemption 은 uncommitted candidate 를 버릴 수 있어야 하고, wake saturation 시 sleep 보다 wake 가 우선한다.

Sleep service 는 scalar FLOP queue 가 아니라 resource vector queue 다.

$$\mathbf{c}_j = (c_{GPU-s}, c_{CPU-s}, c_{HBM-byte-s}, c_{RAM-byte-s}, c_{IO-byte}, c_{net-byte}). \quad (13.2)$$

Mean GPU utilization 이 낮아도 source affinity, validation model, index lock, network link 때문에 P99 job age 가 발산할 수 있다. Queue 안정성은 utilization 한 숫자뿐 아니라 privacy deadline 과 tenant starvation 으로 판정한다.

13.4 State migration roofline

Separate sleep cluster 는 training accelerator utilization 을 높일 수 있지만 source episode, base model, optimizer, adapter candidate, validation trace 를 이동시킨다. Link ℓ 마다 이동 bytes B_ℓ , transfer price/energy p_ℓ , delay T_ℓ , delay value v_{delay} 를 두면

$$C_{move} = \sum_{\ell} (B_\ell p_\ell + T_\ell v_{delay}). \quad (13.3)$$

추가 local execution 과 replication cost 가 C_{move} 보다 작을 때만 compute 를 data 쪽으로 옮긴다. Model/optimizer residency 가 source 보다 훨씬 크면 centralized GPU 가, source-to-delta ratio 와 privacy locality 가 크면 shard-local preprocessing 이 유리할 수 있다.

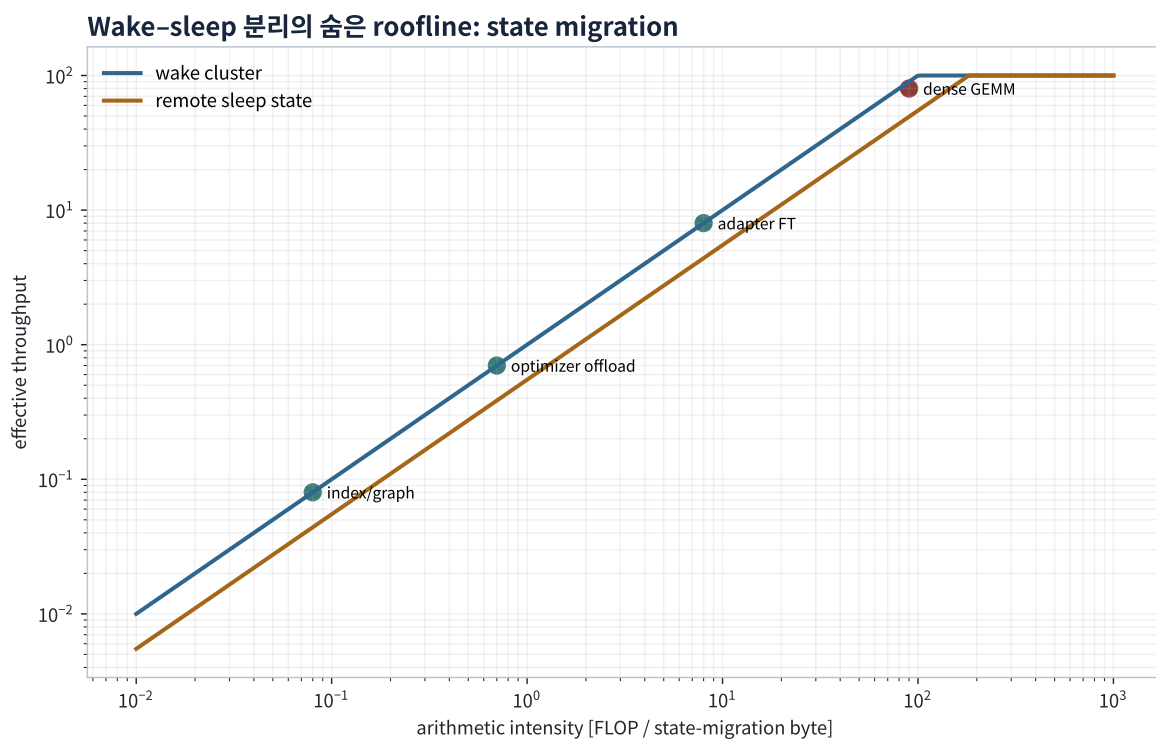


Figure 13.1: Wake/sleep placement 의 state-migration roofline. Arithmetic intensity 가 아니라 유용한 sleep work 대비 이동 state 의 비율과 link bandwidth 가 placement crossover 를 정한다. 표시된 영역은 설계 공간이며 특정 GPU 의 측정 결과가 아니다.

[STC-F011]

출처/제작: Original systems synthesis; SRC-STC-0022, SRC-STC-0059 대체 설명: Shared, separate, shard-local 배치를 state movement 와 compute efficiency 측에서 비교한 roofline 형 지도.

13.5 네 가지 physical placement

Table 13.2: Physical placement option 과 측정해야 할 trade-off

Placement	장점	비용/위험	적합 조건
shared priority fleet	base/model copy 최소, pooling	wake tail contention	sleep 가 작고 preemptible
time-partitioned fleet	idle window 활용	burst 와 timezone mismatch	predictable diurnal load
separate wake/sleep	isolation, training batch 효율	base replica 와 network movement	sustained sleep demand
regional/shard-local hybrid	source locality, privacy	낮은 local compute efficiency, fragmentation	source-to-delta ratio 가 큼

13.6 운영 계약

Minimum viable system 은 (1) append-only event log, (2) exact snapshot, (3) copy-on-write candidate, (4) signed validation result, (5) atomic serving-head swap, (6) delete epoch 와 deny root, (7) version-pinned read, (8) bounded rollback 과 GC owner 를 가져야 한다. Algorithm quality 가 좋아도 이 계약이 없으면 wake request 가 half-published state 를 읽고, deleted source 가 predecessor 에서 부활하며, sleep retry 가 duplicate update 를 만들 수 있다.

핵심 Infra 결론

필요한 분리는 cluster 의 개수가 아니라 responsibility 와 transaction boundary 다. Wake 는 응답과 event capture, sleep 은 candidate construction, memory/control 은 authoritative lineage 와 publication 을 담당한다. Data 이동을 최소화한다는 말은 source 만 보는 것이 아니라 base, optimizer, candidate, validation, predecessor 까지 같은 timestamp 와 link 에서 계측한다는 뜻이다.

Memory-device implication: 알고리즘 이름보다 workload primitive

Memory-device 회사의 기회는 “per-user weight sleep 이 주류가 된다” 는 한 시나리오에 종속되어서는 안된다. External dominant, hybrid, periodic refresh, niche, broad adoption 모두에서 살아남는 traffic 과 control primitive 를 먼저 찾는다. 이 장은 market size 나 제품 우위를 주장하지 않고, 측정해야 할 workload 를 도출한다.

주의 Patent-first publication boundary

Veridraft 는 device 제품화에 관한 10 개 P3 hypothesis 를 internal/patent-first hold 로 분류했다. 따라서 공개 claim ledger 에는 해당 문장과 구현 세부를 실지 않는다. 아래 내용은 이미 공개된 source 에서 도출한 workload class 와 evaluation question 이며, 특정 제품 architecture 나 권리 주장을 승인하는 문서가 아니다. 상세 opportunity matrix 는 별도 내부 review 를 거쳐야 한다.

14.1 왜 inference memory 와 다른 trace 가 생기는가

Fixed-model inference 는 큰 weight 의 read-mostly access, KV append/read, activation traffic 이 중심이다. Wake-sleep memory system 은 여기에 네 종류를 추가한다.

1. **Version traffic:** candidate, predecessor, snapshot, rollback reserve 가 동시에 존재한다.
2. **Index mutation:** vector payload, embedding key, graph edge, temporal validity 가 함께 바뀐다.
3. **Small-artifact fan-out:** user/cohort adapter, router catalog, recurrent state 가 많아지고 hot set 이 시간에 따라 바뀐다.
4. **Lineage control:** source handle, parent/replacement edge, delete epoch, validation attestation 을 low-tail-latency 로 조회한다.

따라서 peak bandwidth 만 높은 장치보다 capacity, endurance, random metadata latency, copy-on-write, atomic generation change, encryption domain, rebuild throughput 의 조합이 중요하다.

14.2 Cross-scenario workload classes

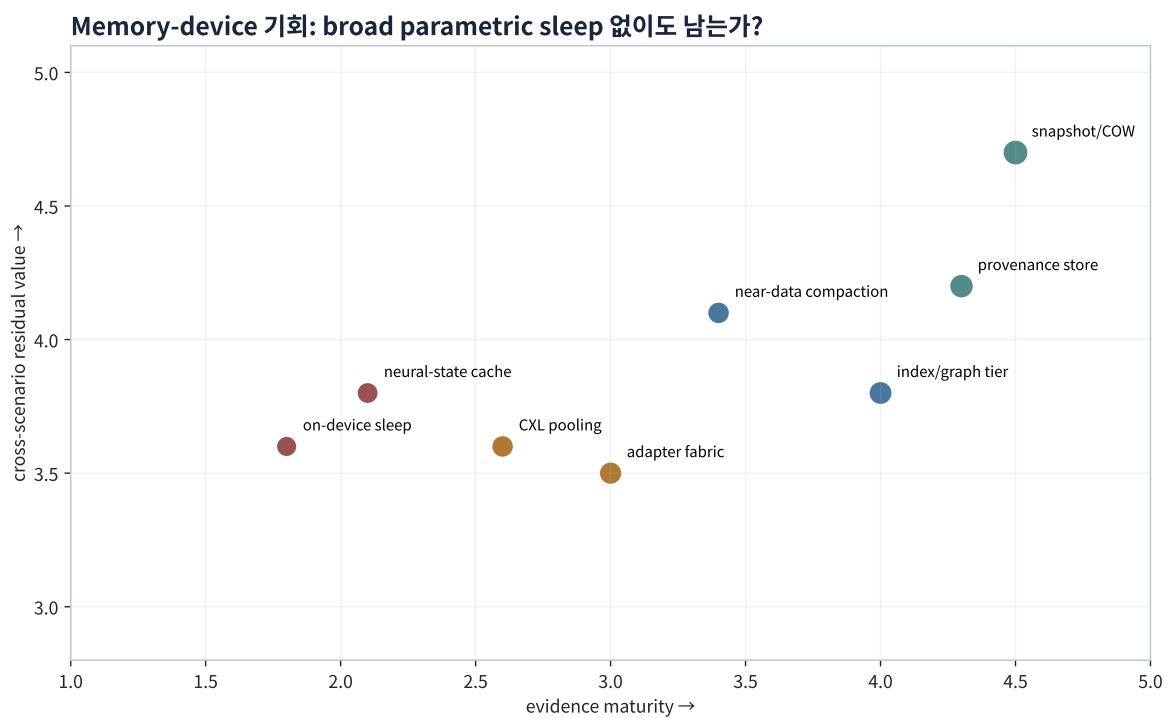


Figure 14.1: 공개 source 에서 관찰 가능한 workload class 를 evidence maturity 와 cross-scenario residual value 의 두 축에 배치한 research map. 좌표는 제품 추천이나 시장 예측이 아니라 benchmark 우선순위를 정하기 위한 정성적 가설이다.

[STC-F012]

출처/제작: Original research map; SRC-STC-0056, SRC-STC-0065 대체 설명: Snapshot, provenance, mutable index, state cache, pooling, local compaction 등 workload 후보를 두 축에 놓은 산점도.

Table 14.1: Device 연구가 먼저 계측할 workload primitive

Class	Trace primitive	시스템 가치 질문	비-STC residual value
provenance episode	append, scan, sparse delete, lineage traversal	rebuild 범위와 audit latency 를 줄이는가	temporal RAG 에도 높음
snapshot/version	COW clone, multi-object publish, predecessor pin	rollback peak bytes/RTO 를 줄이는가	model/index release 에도 높음
mutable index/graph	small random rewrite, compaction, re-embed	write amplification 과 tail latency 는 얼마인가	agent memory 전 반에 높음
state cache	recurrent/KV/adapter load, compatibility lookup	token prefix 외 state reuse 를 회복하는가	RNN/neural memory serving 에 높음
tiered pool	HBM-DRAM-CXL-storage migration	base duplicate 와 state movement 를 동시에 줄이는가	inference 와 refresh 에도 높음
local preprocessing	dedup, cluster, filter, graph maintenance	이동 bytes/energy 절감이 local compute penalty 보다 큰가	data pipeline 에도 중간 - 높음
on-device idle work	burst train/compact, thermal pause, checkpoint	privacy benefit 이 battery/endurance 를 상쇄하는가	local personalization 에 높음
lifecycle telemetry	per-item bytes, age, reuse, rollback, erase	useful retained evidence 를 측정할 수 있는가	모든 memory system 에 높음

14.3 장치 요구사항을 metric 으로 번역하기

Capacity. Live state 만이 아니라 candidate, pinned predecessor, migration duplicate, workspace, replica 를 포함한 $B_{peak,physical}(t)$ 를 측정한다. Snapshot 기능이 있어도 logical clone 과 physical dirty-page growth 를 분리한다.

Endurance. Host write bytes 뿐 아니라 device-level write amplification 을 index/graph rewrite, compaction, garbage collection, secure reclamation 별로 분해한다. Update frequency 가 서로 다른 CMS 또는 adapter tier 는 한 평균 write rate 로 합치지 않는다.

Bandwidth 와 locality. Sequential episode scan, random ANN/graph access, checkpoint delta, adapter page-in, validation readback 을 각각 측정한다. Near-data operator 는 saved link bytes 와 추가 local execution, model residency, result validation cost 를 함께 보고한다.

Tail latency. Manifest, compatibility, lineage, deny-root lookup 은 작은 I/O 지만 wake request 의 critical path 에 있다. Bulk GB/s 가 높아도 P99 metadata lookup 이 길면 serving head 를 안전하게 pin 할 수 없다.

Security 와 recovery. Tenant key boundary, instant deny, physical erase, stale replica discovery, rollback-after-delete 를 fault injection 한다. Encryption throughput 만으로 derived-state deletion 을 증명하지 않는다.

14.4 Device–system 공동 DSE

하드웨어 design space exploration 의 입력은 다음과 같이 workload 에서 직접 온다.

$$\mathcal{D} = \{B_{source}, B_{base}, B_{delta}, B_{index}, B_{recovery}, R/W \text{ pattern}, \tau_{cadence}, Q_{age}, R_v, \text{privacy locality}\}. \quad (14.1)$$

출력은 later-wake utility floor 아래에서 total energy, bytes moved, write amplification, physical peak, P99 wake interference, sleep completion age, rollback/delete time 의 Pareto frontier 다. 특정 장치가 sleep job 을 빨리 끝냈어도 base replica 나 migration 이 전체 비용을 키우면 system win 이 아니다.

14.5 의사결정 순서

1. 공개 trace 와 simulator 로 primitive 를 먼저 검증한다.
2. External-only 와 periodic-refresh 에도 가치가 남는 기능을 Tier 1 으로 둔다.
3. Hybrid/broad scenario 에만 가치가 큰 state/adapter 기능은 leading indicator 를 조건으로 둔다.
4. Patentability, freedom-to-operate, security review 가 끝나기 전에는 public claim 으로 승격하지 않는다.
5. 제품 KPI 를 bandwidth 가 아니라 useful retained evidence per lifecycle resource 로 연결한다.

핵심 Device 관점의 핵심

가장 강건한 기회는 “sleep training 전용 memory” 라는 이름이 아니라, versioned derived state 를 많이만들고 검증하고 되돌리는 시스템의 memory primitive 다. Parametric sleep 이 지연되어도 provenance, snapshot, temporal index, heterogeneous state caching 과 lifecycle telemetry workload 는 남는다.

Falsifiable benchmark 와 experiment roadmap

이 분야에 필요한 다음 산출물은 더 큰 metaphor 가 아니라 matched lifecycle experiment 다. 같은 wake events 와 future queries 를 주고, exact external retention, external compaction, latent/KV compilation, user-local adapter, deliberate forgetting, past-only hybrid routing 중 무엇이 가장 나은지를 측정해야 한다. Null result 와 boundary 를 계획에 포함해야 negative paper 도 가치가 있다.

반증 가능한 research → system → device agenda

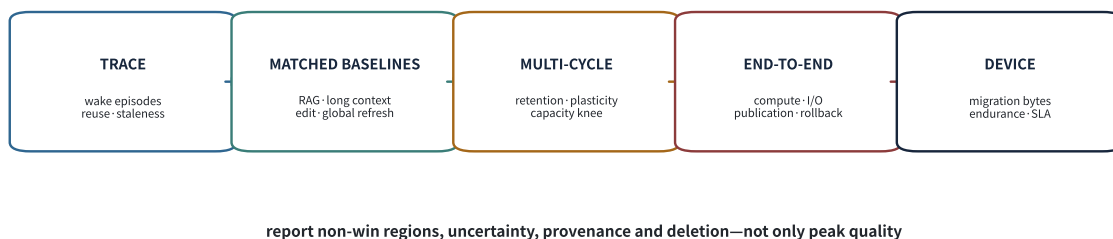


Figure 15.1: Falsifiable research agenda. Evidence stream, candidate construction, lifecycle validation, repeated-cycle evaluation, scaling fit 을 하나의 provenance chain 으로 연결한다.
[STC-F015]

출처/제작: Original synthesis; SRC-STC-0036, SRC-STC-0050, SRC-STC-0056 대체 설명: Matched methods, cycle probes, systems/governance metrics, held-out scaling test 가 순차적으로 이어지는 roadmap.

15.1 독립 단위와 cycle

최상위 independent unit 은 user/stream 이다. Cycle 과 probe 를 독립 sample 처럼 세어 significance 를 부풀리지 않는다. 한 cycle 은 다음 순서를 가진다.

wake capture → pre-sleep probe → immutable snapshot → candidate
→ validate/publish → post-sleep probe → delayed next-wake probe.

Default horizon 은 128 cycles, sensitivity 는 $C \in \{8, 32, 128, 512\}$, retrieval lag 는 가능한 경우 $\{1, 8, 32, 128\}$ cycles 로 둔다. 단일 sleep 후 정확도는 lifelong behavior 를 지지하지 못한다.

15.2 Observable 과 oracle 을 물리적으로 분리한다

Method 가 보는 observable stream 에는 event payload, 현재 confidence, privacy scope, 알려진 supersession/delete, past reuse, resource/queue state 만 있다. True future reuse, future

contradiction, future deletion, poison truth, ground-truth dependency, clairvoyant destination 은 oracle ledger 에 따로 둔다. 파일, schema, API permission 과 process boundary 로 누출을 막는다.

Train 은 learned router/compiler 를 fit 하고, CALIBRATION 은 threshold, cadence, baseline hyperparameter 를한 번 선택하며, TEST 는 G2 freeze 뒤 한 번만 실행한다. TEST family identity 나 future reuse 로 configuration 을바꾸는 mutation test 가 통과해야 한다. Dream-generated probe 도 evaluation query 가 아니라 derived training artifact 이므로 generator/prompt/input/output/filter digest 를 남긴다.

15.3 Event 와 query family

Table 15.1: 독립적으로 변화시킬 event factor 와 주요 failure

Family	Controlled variables	잡아야 할 failure
stable exact fact	entropy, surface length, reuse, lag	semantic compression 이 exact detail 을 지움
volatile fact/preference	validity interval, correction hazard	stale overwrite, historical loss
procedure/skill	reusable/one-off, environment version	vague summary, non-executable memory
relation/graph failure feedback	alias, temporal edge, hop count wrong action, correction, avoidance	entity collision, invalid edge success-only bias
privacy/delete	consent expiry, dependency fan-out	tombstone-only delete
poison/adversarial	injection, conflicting frequency, delayed label	poison amplification
one-off distractor	salience, payload size, zero reuse	unnecessary consolidation

Reuse, volatility, poison, deletion, payload size 와 memory pressure 는 서로 독립적으로 변화시킨다. Query 는 exact, semantic, temporal, relational, procedural, negative/failure, calibration, deletion family 로 나눈다. Acquisition query, exact repeat, held-out paraphrase, held-out composition, correction probe, unrelated control 을동일 지표로 합치지 않는다. “기억한 문장을 그대로 다시 답했다” 와 “새 상황에 procedure 를 적용했다” 는다른 능력이다.

15.4 Confirmatory methods 와 resource envelope

Primary deployable family 는 raw external, compressed external, latent/KV compiler, user-parametric LoRA, static mixture, past-only hybrid router 여섯 개다. Clairvoyant oracle 은 heterogeneity 의 upper bound 일 뿐 deployable method 가 아니다. Long-context no-memory lower bound 와 unbounded full-history upper bound 는 diagnostic 으로 남긴다.

모든 method 는 다음 upper bound 를 공유한다.

- live token, active byte, durable byte, total incremental peak-state byte,

- physical HBM/fleet peak, sleep FLOPs, peak accelerator,
- 동일한 information cutoff, privacy 와 delete contract.

Search/tuning, router, judge, embedding, synthetic data, optimizer, answer-reachable KV/recur-rent state, index rebuild, raw archive, lineage, rollback reserve 와 transient workspace 를 모두 charge 한다. 사용하지 않은 budget 은 낭비하도록 강제하지 않고 그대로 보고한다.

Reuse $R \in \{0, 1, 4, 16\}$ 와 active-capacity ratio $\phi \in \{1, 1/4, 1/16\}$ 을 교차한다. Schedule/operator factorial 은 inline versus deferred 와 identity versus semantic consolidation 을 destination external 로 고정해, execution timing 의 효과와 추가 정보를 더본 효과를 분리한다.

15.5 Metric stack

Table 15.2: 한 결과표에 함께 있어야 할 metric stack

Layer	Required metrics
learning	lifetime utility AUC, current utility, acquisition, retention/forgetting, forward transfer, memory half-life
semantic	exact/semantic/temporal/relational/procedural, composition, relevance, calibration, false memory
memory path	writer/retriever/reader error, route/merge compatibility, coverage, faithfulness, detail loss, stale conflict
robustness	poison amplification, cross-user/task interference, rare-tail survival, recursive-depth distortion
governance	access deny, physical removal, lineage invalidation, key expiry, behavioral residual, rollback success/time
serving	TTFT/ITL/task P50/P95/P99, throughput, cache/adaptor hit, load/swap, fallback
sleep service	throughput, queue age, deadline miss, retry, validation/publish/rollback cost
resources	pre/post/wake/sleep/validation FLOPs, retained/peak bytes, bytes moved by link, write amplification, energy
compatibility	base/tokenizer/embedding version, artifact route, rebase and rebuild cost

Primary quality endpoint 은 cycle 과 pre/post/delayed probe block 에 동일 가중치를 주는 lifetime utility AUC 다. Energy-per-correct 는 frozen accuracy floor 위에서만 계산한다. Delete score 는 access denial 과 physical removal, derived invalidation, behavioral residual 을 한 binary 로 합치지 않는다.

15.6 통계 · 재현성과 claim-down rule

모든 method 는 같은 stream, event order, query order, resource tuple 을 받는다. User/stream 을 paired resampling unit 로 사용하고 workload-family×seed-family block 을 유지한다. Hardware placement 와 method order 는 randomize 또는 block 한다. Outcome 을 본 뒤 stopping point, comparator, hypothesis family 를 바꾸지 않는다.

Run artifact 는 source/version → question/hypothesis → config/data/code/environment → raw checksum → result → claim → figure/table → manuscript sentence 를 연결한다. Timeout, OOM, deadline miss, cap violation 은 missing data 가 아니라 consumed work 를 charge 한 RESOURCE_FAILURE 로 처리한다.

다음 경우 claim 을 낮춘다: effect 가 materiality margin 보다 작음, equivalence interval 이 margin 을 걸침, simulation 으로 absolute hardware latency 를 말함, archive 를 남기고 total compression 이라 함, behavioral forgetting 을 physical deletion 이라 함, post-hoc arm 을 추가함. Minimum publishable result 는 hybrid 가실패해도 deterministic leakage-tested stream, matched methods, complete resource ledger, retention/acquisition 분리와 잘 추정된 crossover·knee·null 중 하나를 남기는 것이다.

핵심 실험의 최종 질문

“Sleep 이 정확도를 높였는가?” 가 아니라 “동일한 evidence 와 lifetime budget 에서, 어떤 admission, operator, destination, cadence 가 later-wake utility 와 governance 의 Pareto frontier 를 움직였는가?” 를 묻는다.

Limitations, negative evidence와 falsifiers

이 Study 의 결론은 2026-08-05 공개 evidence 에 대한 조건부 synthesis 다. Product 내부 trace, 비공개 deployment incident, 실제 device workload 가 없으며, 논문 간 model·task·budget 이 달라 meta-analysis 를수행하지 않았다. Original figure 의 좌표와 scenario 는 측정 결과가 아니라 reasoning aid 다.

16.1 문헌 자체의 한계

명시적 parametric sleep 연구는 direct mechanism evidence 지만 model scale, task diversity, cycle count 와 production governance 가 제한된다. Product documentation 은 background phase 의 존재를 보여도 strongest alternative 대비 causal advantage 를 보여 주지 않는다. Continual-learning precursor 는 forgetting 을 줄이는방법을 제공하지만 LLM agent 의 deletion, state migration, future reuse economics 를 직접 측정하지 않는다.

DANN/SSRN record 는 동결 source 에서 independent verification 에 필요한 method 와 result 를 충분히 공개하지않아 load-bearing evidence 로 사용하지 않았다.^[STC-C047] (Tutuncuoglu 2025) 제목이나 abstract 가 “zero forgetting” 을 말해도 full method, data, comparator, statistics, artifact 가 없으면 central claim 을지탱할 수 없다. 이 판단은 논문이 틀렸다는 결론이 아니라 증거 접근성에 대한 claim-down 이다.

16.2 Negative evidence 가 가리키는 구조적 위험

Table 16.1: 실패 mode 와 필요한 control

Risk	관찰/이론적 근거	필요한 control
behavioral unreachability	sequential fact 가 weights 에 남아도 normal query 에서 접근 불가	direct, prompted, compositional reachability 를 분리
catastrophic interference	new update 가 old task 와 plasticity 를 손상	old/new/safety slice, replay/regularization/isolation ablation
recursive-data collapse	synthetic-only generations 가 rare tail 을 잃을 수 있음	canonical anchor, recursive depth, common/tail distortion
false merge/compaction	summary 가 exact, temporal, negative detail 을 지움	source spans, faithfulness, exact/temporal query, rebuild
memory poisoning	persistent record 가 후속 behavior 를 조종	provenance, quarantine, poison canary, least privilege
delete illusion	tombstone 뒤 summary, index, cache, adapter 에 영향 잔존	lineage fan-out, deny root, physical removal, residual probe

Risk	관찰/이론적 근거	필요한 control
state fan-out	user/cohort artifact 와 catalog 가 base saving 을 상쇄	total state, hot residency, route/load latency
queue/SLA failure	평균 utilization 전에도 affinity-validation 이 P99 age 를 악화	multi-resource queue, deadline, wake preemption
version churn	base/tokenizer/embedding 변경이 artifact 를 무효화	compatibility digest, rebase/rebuild cost

Failure mode마다 lifecycle control을 붙인다

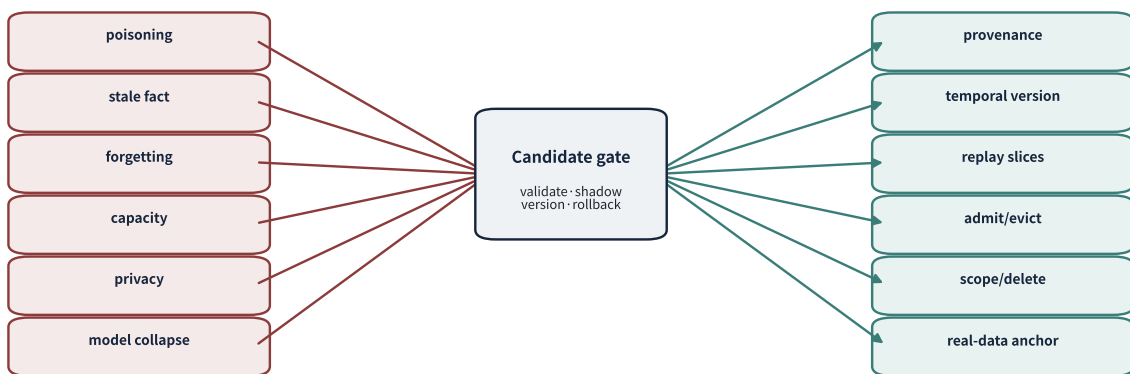


Figure 16.1: Wake capture 에서 deletion 까지 이어지는 주요 failure mode 와 lifecycle control. 한 단계의 accuracy gate 만으로는 poison, stale publish, rollback resurrection, residual delete 를 막을 수 없다.

[STC-F013]

출처/제작: Original synthesis; SRC-STC-0042, SRC-STC-0053 대체 설명: Admission, candidate, publish, serve, delete 단계마다 poison-interference-race-residual 과 대응 control 을 연결한 흐름도.

16.3 결론을 뒤집거나 좁힐 구체적 falsifier

F1—새 phase claim. 동일 transformation, information cutoff, destination, total resource 를 맞췄을 때 deferred 와 synchronous 실행의 later-wake quality, wake-SLA, batching 차이가 모두 사라지면 “새 scientific axis” 는 scheduling label 로 좁혀야 한다.

F2—hybrid recommendation. External-only 또는 direct parametric-only 가 stable skill 과 volatile fact 모두에서 utility, cost, deletion, rollback, portability 를 지배하면 hybrid default 를 폐기한다.

F3—valid-reuse hypothesis. Reuse 와 query predictability 를 넓게 sweep 해도 STC gain 과 interaction 이 없거나 predictor error 가 top- k yield 를 없애면 reuse-based promotion 이론을 기각한다.

F4—capacity knee. Repeated stream 에서 behavioral retention/plasticity 가 physical occupancy 와 같이 부드럽게 변하고 앞선 knee 가 재현되지 않으면 별도 behavioral knee hypothesis 를 기각한다.

F5—canonical anchoring. Synthetic/summary replay 와 source-anchored replay 의 rare-tail survival 이 matched cost 에서 차이가 없으면 canonical anchor 의 안정성 주장을 낮춘다.

F6—dedicated sleep infra. Shared priority fleet 가 모든 realistic trace 에서 더 낮은 physical peak, 이동 bytes, P99 wake latency 와 job age 를 동시에 보이면 separate sleep cluster 필요성을 기각한다.

F7—device relevance. Background memory job 이 commodity CPU/object-store metadata 수준에 머물고 retained bytes, rewrite, movement, tail-SLA 가 장치 선택에 영향을 주지 않으면 새로운 memory workload 라는 주장을 좁힌다.

16.4 Coverage 와 saturation 의 한계

Search cluster 는 date-bounded snapshot 이며 five active frontier 와 five measurement gap 을 명시적으로 남겼다. 특히 Google, Meta, Microsoft 의 research output 은 넓지만 public production per-user weight sleep 의 부재를 “그 회사가 하지 않는다” 는 negative fact 로 바꾸지 않는다. 공개하지 않았을 가능성과 아직 측정하지 않은 possibility 가 있기 때문이다.

Figure 는 권리 검토된 original redraw 만 사용했으며 source paper figure 를 직접 복제하지 않았다. 이 선택은 copyright risk 를 줄이지만 원 실험 plot 의 numeric fidelity 를 대신하지 않는다. 독자는 claim ledger 와 primary source 로 수치를 확인해야 한다.

16.5 본 논문이 주장하지 않는 것

- Biological NREM/REM stage 가 AI operator 에 one-to-one 으로 대응한다는 주장,
- 더 많은 sleep FLOPs 가 항상 utility 를 높인다는 universal monotone law,
- weights 가 external memory 보다 본질적으로 더 장기적이라는 주장,
- 공개 product lifecycle 가 comparative scientific optimality 를 입증한다는 주장,
- simulation 이 특정 GPU-memory device 의 latency, energy, endurance 를 측정했다는 주장,
- P3-held device hypothesis 가 publication 또는 patent review 를 통과했다는 주장.

주의 가장 중요한 한계

현재 논문은 measurement program 과 conditional strategy 를 제시한다. STC 의 universal dominance 나 broad parametric deployment 를 관측한 논문이 아니다. 이 구분을 유지해야 향후 negative result 가 연구 실패가 아니라 이론의 경계를 좁히는 결과가 된다.

Conclusion: 다음 학습 축의 조건과 연구 순서

본 연구의 답은 단순한 찬반이 아니다. Sleep-time compute 은 release-time pre/post-training 과 latency-critical test-time learning 사이에, accumulated wake experience 를 reusable later-wake state 로 바꾸는 deferred lifecycle budget 을 추가한다. 이 기능적 축은 이미 external background synthesis 에서 product evidence 를 갖는다. 그러나 broad per-user foundation-weight sleep 이 주류가 되거나 보편 scaling law 를 가진다는 증거는 없다.

17.1 처음의 질문에 대한 직접 답

무슨 문제를 푸는가? Static deployment 와 persistent agent experience 사이의 간극, 반복되는 long context 비용, cross-session strategy learning, freshness/contradiction, finite-memory compaction, wake latency 와 learning throughput 의 충돌을 푼다. 하나의 method 가 이 문제를 모두 풀지는 않는다.

Sleep 이 최선인가? One-off document, exact volatile fact, unpredictable query 에는 long context, RAG 와 temporal external memory 가 더 강하다. Stable high-reuse procedure, multi-episode pattern, background conflict resolution 에서는 deferred transformation 이 유망하다. Continual learning 에서는 sleep 이 replay, regularization, isolation, validation 을 실행하는 phase 를 제공할 뿐 capacity theorem 을 없애지 않는다.

Parametric 인가 external 인가? Binary choice 가 아니다. External source-of-record 에 canonical evidence 와 provenance 를 보존하고, calibrated reuse 와 stability 가 충분한 subset 만 reversible parametric artifact 로 compile 하는 hybrid 가 현재 가장 방어적이다. Changing fact, deletion-sensitive episode, low-reuse item 은 external 에 남는다.

업계와 학계는 어디로 가는가? 업계는 external synthesis, provenance, control 과 background job 으로, 학계는 learned memory writer/router, neural fast state, continual-learning mechanism, compaction theory 와 benchmarks 로 수렴한다. Lifecycle 이 mainstream 이 되어도 이름은 dreaming, reflection, synthesis 또는 memory 로 남을 수 있다.

Scaling law 는 무엇인가? 지금은 법칙이 없다. 먼저 complete state/work identity, reuse 와 cadence 의 restricted break-even, capacity knee 와 state-migration crossover 를 측정한다. 그다음 evidence quality, valid reuse, cycle count, recursive depth, medium, hardware 를 교차한 held-out predictive surface 가 반복되면비로소 제한된 scaling law 를 말할 수 있다.

Memory-device 회사의 기회는 무엇인가? 특정 parametric sleep adoption 에 베팅하기보다 versioned derived state, provenance, temporal index, snapshot/rollback, heterogeneous state cache, state-local processing, lifecycle telemetry 의 실제 trace 를 공동 측정해야 한다. 상세 제품 hypothesis 는 patent-first review 뒤에 공개한다.

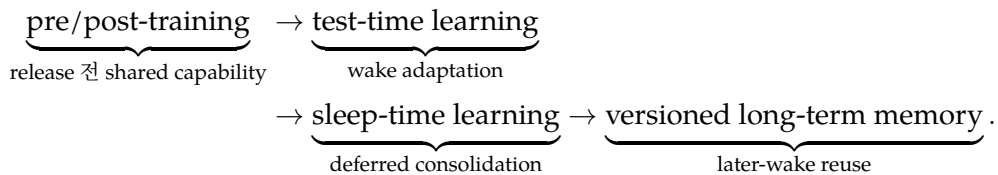
17.2 권장 연구 순서

1. **External lifecycle baseline:** versioned raw/summary/vector/graph 와 exact delete/rollback 을 먼저 구현해 source-of-truth 와 metric 을 고정한다.

2. **Matched benchmark:** 동일 event/query/resource 에서 raw, compressed, latent, LoRA, static mixture, past-only router 를 128+ cycles 비교한다.
3. **Reuse/capacity sweep:** R , volatility, active capacity, recursive depth, cadence 를 독립적으로 변화시켜 crossover 와 knee 를 찾는다.
4. **Selective promotion:** stable procedural subset 에만 versioned adapter/latent compile 을 허용하고 rebase, demotion, delete fault 를 주입한다.
5. **Trace-driven infra DSE:** shared, time-partitioned, separate, shard-local placement 에서 base residency 와 모든 state-migration bytes 를 계측한다.
6. **Device co-design:** public trace 로 capacity, endurance, tail latency, COW, secure reclamation 의 sensitivity 를 검증한 뒤 product research 로 승격한다.

17.3 마지막 framing

앞으로의 학습을 세 phase 로 보는 framing 은 유용하다.



다만 화살표는 일방향이 아니다. Delete 와 correction 은 long-term memory 에서 derived state 로 역전파되고, base release 는 adapter 와 index 를 재검증하며, wake failure 는 다음 sleep candidate 를 만든다. 미래의 scaling object 는 checkpoint 하나가 아니라 여러 clock 과 memory medium, provenance 와 transaction 을 가진 system 일 가능성이 높다.

핵심 최종 verdict

Sleep-time compute 은 promising 한 새 lifecycle 축이지만, 현재의 승자는 broad weight update 가 아니라 external background consolidation 이다. 가장 유망한 다음 단계는 external source-of-record 위의 selective, reversible promotion 이며, 그 가치는 sleep FLOPs 가 아니라 valid later-wake reuse per complete lifecycle resource 로 검증해야 한다.

Canonical claim – evidence ledger

본문의 파란 claim ID 는 이 표의 canonical English statement 와 source locator 로 연결된다. 이 표는 번역문이 아니라 Veridraft gate 를 통과한 frozen assertion surface 다.

Table 1: Veridraft-gated public claims (source freeze 2026-08-05)

ID	Type / confidence	Canonical claim	Evidence locator
STC-C001	direct fact high	The date-frozen evidence audit contains 73 primary papers, official releases, documentation pages, and tagged repositories.	SRC-STC-0014: source registry audit; entry SRC-STC-0014
STC-C002	direct fact high	By 2026-08-05, at least two public production systems described explicit background memory synthesis: OpenAI Dreaming and Mem0 Dream.	SRC-STC-0052: official release, opening and How dreaming works SRC-STC-0063: official docs, Overview and How Dream works
STC-C003	direct fact high	The 12-cluster search closes only as a bounded date snapshot: 2 clusters are bounded, 5 remain active frontiers, and 5 retain measurement gaps.	SRC-STC-0036: paper abstract and §5 limitations
STC-C004	direct fact high	A balanced 15-paper translation corpus spans explicit sleep, biological and computational precursors, continual learning, external memory, neural memory, capacity, and negative evidence.	SRC-STC-0011: full paper, title through Discussion
STC-C005	synthesis high	Sleep-time compute is best defined by lifecycle, not by metaphor: computation is deferred beyond the latency-critical response and changes reusable state for later wake episodes.	SRC-STC-0014: abstract; §1; Fig. 1 SRC-STC-0052: official release, How dreaming works SRC-STC-0063: official docs, Synthesis mode
STC-C006	synthesis high	Update timing and memory medium are orthogonal axes: a sleep phase may update weights, adapters, recurrent state, text/vector stores, graphs, or mixtures.	SRC-STC-0014: §2, sleep-time agent architecture SRC-STC-0021: §2-§3, self-modification and consolidation SRC-STC-0016: §2-§3, temporal graph memory
STC-C007	direct fact high	OpenAI Dreaming is production background synthesis over external user memory, not disclosed per-user foundation-weight training.	SRC-STC-0052: official release, opening through How dreaming works
STC-C008	direct fact high	Mem0 Dream performs scheduled background synthesis with provenance-preserving, additive outputs; merge and supersede are distinct ingest-time operations.	SRC-STC-0063: official docs, Overview; Modes; Operational behavior SRC-STC-0064: tag v2.0.13, dream-gate.ts
STC-C009	direct fact high	Letta’s tagged sleeptime group schedules an asynchronous post-response agent that revises durable memory outside the response-critical path.	SRC-STC-0059: tag 0.16.8, sleep-time_multi_agent_v4.py
STC-C010	direct fact high	Zep and Graphiti represent external memory as temporally versioned episodes, entities, and relations rather than as user-specific model-weight updates.	SRC-STC-0016: abstract; §2 Architecture; §3 Temporal KG SRC-STC-0065: commit aab852d, README overview SRC-STC-0066: official docs, Episodes
STC-C011	author claim high	MemGPT frames long-term agent memory as an operating-system-style hierarchy that pages information between constrained context and external storage.	SRC-STC-0013: abstract; §2 Virtual Context Management; Fig. 1
STC-C012	direct fact high	ReasoningBank stores reusable reasoning strategies extracted from experience; the public method is an external strategy memory rather than broad online weight consolidation.	SRC-STC-0043: abstract; §2-§3 SRC-STC-0054: official Google Research post, Method

다음 쪽에 계속

ID	Type / confidence	Canonical claim	Evidence locator
STC-C013	synthesis medium	Meta’s personalized-agent and HyperAgents work emphasizes feedback-conditioned agent behavior, program memory, and archives; public evidence does not establish fleet-scale per-user sleep updates to foundation weights.	SRC-STC-0057: official Meta paper page, abstract SRC-STC-0058: official Meta paper page, abstract
STC-C014	direct fact high	Microsoft STATE-Bench evaluates agent memory across state tracking, adaptation, and efficiency, but is a benchmark rather than a disclosed sleep-training product.	SRC-STC-0056: official Microsoft post, benchmark dimensions SRC-STC-0067: commit 4efcbf2, README tasks and metrics
STC-C015	synthesis high	Language Models Need Sleep is direct evidence for parametric sleep: the model learns to self-modify and consolidate memories using replay-like training, but the reported regime remains research-scale rather than production deployment proof.	SRC-STC-0021: abstract; §2 Method; §4 Experiments; §6 Limitations
STC-C016	author claim high	In a two-task spiking-network setting, interleaved sleep-like replay formed a joint synaptic representation and reduced catastrophic forgetting.	SRC-STC-0011: abstract; Results, Figs. 2-7; Discussion
STC-C017	author claim high	A hippocampus-neocortex model used autonomous offline dynamics and alternating NREM/REM regimes to integrate new information while protecting older knowledge.	SRC-STC-0010: abstract; Fig. 1; Simulation 2; Fig. 4
STC-C018	author claim high	Perturbed and Adversarial Dreaming separates wake, NREM-like perturbed replay, and REM-like adversarial dreaming to learn cortical representations.	SRC-STC-0009: abstract; Results; Figs. 1-9; Discussion
STC-C019	author claim high	Deep generative replay is a strong non-sleep-named baseline: a generator replays pseudo-data while the solver learns new data, reducing catastrophic forgetting.	SRC-STC-0003: abstract; §2 Deep Generative Replay; Algorithm 1
STC-C020	author claim high	Elastic Weight Consolidation protects parameters judged important to earlier tasks through a Fisher-weighted quadratic penalty, trading plasticity for stability.	SRC-STC-0002: §2; Eq. 3; experiments
STC-C021	author claim high	Gradient Episodic Memory constrains updates using stored examples so that loss on prior tasks does not increase under the local constraint.	SRC-STC-0004: §2; Eqs. 3-10; Algorithm 1
STC-C022	author claim high	Progress & Compress alternates an active column with a consolidation phase that compresses knowledge into a reusable knowledge base, providing a direct periodic-refresh alternative to explicit sleep framing.	SRC-STC-0007: abstract; §2 Progress; §3 Compress; Fig. 1
STC-C023	synthesis high	Titans learns a neural memory state at test time using surprise-driven updates; this is wake-path fast-state learning, not by itself deferred cross-session sleep.	SRC-STC-0017: §2 Neural Long-Term Memory; Eqs. 7-13; Fig. 1
STC-C024	synthesis high	Nested Learning/HOPE treats optimization processes at multiple update frequencies as nested memory levels, supplying a useful cadence model but not public evidence of deployed asynchronous sleep consolidation.	SRC-STC-0020: §2 Nested Learning; §4 HOPE; CMS frequency schedule
STC-C025	author claim high	Memory Caching identifies a serving problem for recurrent memory models: reusable state has to be materialized, keyed, cached, and shared to recover prefix-cache-like efficiency.	SRC-STC-0022: abstract; §2 Memory Caching; §4 Serving Experiments
STC-C026	author claim high	Retrieval-augmented generation keeps mutable knowledge outside model weights and conditions generation on retrieved passages, making it a strong update-cost and governance baseline.	SRC-STC-0031: abstract; §2 Method; Fig. 1
STC-C027	synthesis high	LoRA constrains adaptation to low-rank weight deltas, reducing trainable parameter and optimizer-state cost but not eliminating sequential interference or lifecycle governance.	SRC-STC-0030: abstract; §4 Method; Eq. 3 SRC-STC-0036: §3 sequential fact-learning results
STC-C028	synthesis high	ROME localizes and edits factual associations in transformer MLP weights, showing targeted parametric change is possible but not proving stable unbounded continual accumulation.	SRC-STC-0032: abstract; §3 Causal Tracing; §4 ROME
STC-C029	synthesis high	MEMIT extends model editing to many memories, but mass editing remains a bounded batch intervention rather than a demonstrated lifelong sleep loop.	SRC-STC-0033: abstract; §3 MEMIT; scaling experiments
STC-C030	synthesis high	Generative Adapter compiles context into parameters in one forward pass, suggesting a latent-memory path whose value depends on reuse amortizing compilation and version-management cost.	SRC-STC-0027: abstract; §3 Generative Adapter; experiments

다음 쪽에 계속

ID	Type / confidence	Canonical claim	Evidence locator
STC-C031	synthesis high	Self-Adapting Language Models generate their own adaptation data and update rules, bridging wake-time learning and later consolidation while introducing recursive-data quality risks.	SRC-STC-0029: abstract; §2-§3 self-edits SRC-STC-0041: model-collapse results
STC-C032	author claim high	Sequentially learned facts can remain encoded in weights yet become behaviorally unreachable; retention must therefore be measured by access and composition, not only by parameter storage.	SRC-STC-0036: abstract; §4 Results; §5 Discussion
STC-C033	synthesis high	Static memorization capacity measurements do not directly determine safe continual-learning capacity because sequential access, interference, and governance add independent constraints.	SRC-STC-0034: abstract; capacity experiments SRC-STC-0036: §4-§5
STC-C034	synthesis high	A rate-distortion view makes memory compaction an explicit utility-budget tradeoff, but its proposed frontier is not yet a universal empirical scaling law for agents.	SRC-STC-0035: abstract; §2 Rate-Distortion Formulation; limitations
STC-C035	author claim high	Recursive training on generated data can collapse distribution tails when real-data support is lost, so sleep-generated replay cannot be treated as costless fresh data.	SRC-STC-0041: main results; Figs. 1-3; Methods
STC-C036	author claim high	Model collapse is not inevitable when real and synthetic data are accumulated under appropriate conditions; the relevant scaling variable is recursive depth and real-data anchoring, not a binary synthetic-data flag.	SRC-STC-0069: abstract; theoretical results; experiments
STC-C037	synthesis high	External memories create an attack surface: poisoned records can steer agent behavior while remaining hard to notice, making provenance and revocation first-class design requirements.	SRC-STC-0042: abstract; threat model; attack results
STC-C038	synthesis high	As of the freeze date, public industry evidence is strongest for external-memory sleep and background synthesis, not broad per-user foundation-weight sleep.	SRC-STC-0052: official OpenAI Dreaming release SRC-STC-0063: official Mem0 Dream docs SRC-STC-0059: tagged Letta sleep-time implementation SRC-STC-0065: tagged Graphiti repository
STC-C039	synthesis medium	Broad per-user parametric sleep remains technically immature because public evidence does not jointly establish continual retention, rollback, deletion, security, and fleet-level serving economics.	SRC-STC-0021: §4 experiments and §6 limitations SRC-STC-0036: §4-§5 sequential-learning limits SRC-STC-0042: memory-poisoning threat model
STC-C040	synthesis medium	The most credible medium-term architecture is hybrid: keep an auditable external system of record, then selectively promote high-reuse, stable knowledge into reversible adapters or bounded parametric state.	SRC-STC-0052: external synthesis lifecycle SRC-STC-0027: parameter compilation method SRC-STC-0030: low-rank adapter method SRC-STC-0066: episode provenance
STC-C041	synthesis high	No source in the frozen corpus establishes a universal sleep-time scaling law; current equations should be labeled accounting identities, break-even models, fit candidates, or untested hypotheses.	SRC-STC-0034: static memorization-capacity experiments SRC-STC-0035: rate-distortion proposal SRC-STC-0050: memory-statistics tradeoff theory
STC-C042	synthesis high	The lifecycle may become mainstream without the term 'sleep-time compute' becoming the dominant label; current systems use dreaming, reflection, synthesis, memory, self-evolution, and consolidation language.	SRC-STC-0052: title and product terminology SRC-STC-0054: title and product terminology SRC-STC-0060: reflection-launcher terminology SRC-STC-0063: Dream terminology

다음 쪽에 계속

ID	Type / confidence	Canonical claim	Evidence locator
STC-C043	hypothesis medium	A useful candidate scaling variable is valid reusable evidence per unit sleep cost, not raw memories processed or raw sleep FLOPs.	SRC-STC-0043: strategy selection and reuse results SRC-STC-0051: RL over memory objective SRC-STC-0035: utility-budget formulation
STC-C044	hypothesis medium	The optimal sleep cadence should balance staleness, batching efficiency, interference, and wake-SLA impact; neither continuous updating nor arbitrarily long batching is generally optimal.	SRC-STC-0020: multi-frequency update framework SRC-STC-0063: scheduled synthesis cadence SRC-STC-0022: serving and state-reuse costs
STC-C045	hypothesis medium	Finite memory systems should exhibit a capacity knee where marginal consolidation utility falls and interference, compaction, or deletion cost rises sharply.	SRC-STC-0034: memorization-capacity curves SRC-STC-0035: rate-distortion frontier SRC-STC-0036: sequential reachability degradation
STC-C046	synthesis high	For derived memory, the external source-of-record plus provenance graph is a stronger governance basis than treating updated weights as the sole authoritative record.	SRC-STC-0053: official memory controls SRC-STC-0063: provenance and non-destructive behavior SRC-STC-0066: episode provenance and temporal facts
STC-C047	direct fact high	The DANN/SSRN record is not load-bearing evidence because the frozen source exposes insufficient methods and results for independent verification.	SRC-STC-0039: SSRN abstract and metadata landing page

참고문헌

- Behrouz, Ali, Farnoosh Hashemi, Adel Javanmard, and Vahab Mirrokni (2026a). *Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories*. Frozen source SRC-STC-0021; type=paper; accessed=2026-08-05. arXiv: 2606.03979. URL: <https://arxiv.org/abs/2606.03979v2> (visited on 08/05/2026).
- Behrouz, Ali, Zeman Li, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni (2026b). *Memory Caching: RNNs with Growing Memory*. Frozen source SRC-STC-0022; type=paper; accessed=2026-08-05. arXiv: 2602.24281. URL: <https://arxiv.org/abs/2602.24281v1> (visited on 08/05/2026).
- Behrouz, Ali, Meisam Razaviyayn, Peilin Zhong, and Vahab Mirrokni (2025). *Nested Learning: The Illusion of Deep Learning Architectures*. Frozen source SRC-STC-0020; type=paper; accessed=2026-08-05. arXiv: 2512.24695. URL: <https://arxiv.org/abs/2512.24695v1> (visited on 08/05/2026).
- Behrouz, Ali, Peilin Zhong, and Vahab Mirrokni (2024). *Titans: Learning to Memorize at Test Time*. Frozen source SRC-STC-0017; type=paper; accessed=2026-08-05. arXiv: 2501.00663. URL: <https://arxiv.org/abs/2501.00663v1> (visited on 08/05/2026).
- Chen, Tong, Hao Fang, Patrick Xia, Xiaodong Liu, Benjamin Van Durme, Luke Zettlemoyer, Jianfeng Gao, and Hao Cheng (2024a). *Generative Adapter: Contextualizing Language Models in Parameters with A Single Forward Pass*. Frozen source SRC-STC-0027; type=paper; accessed=2026-08-05. arXiv: 2411.05877. URL: <https://arxiv.org/abs/2411.05877v1> (visited on 08/05/2026).
- Chen, Zhaorun, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li (2024b). *AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases*. Frozen source SRC-STC-0042; type=paper; accessed=2026-08-05. arXiv: 2407.12784. URL: <https://arxiv.org/abs/2407.12784v1> (visited on 08/05/2026).
- Colaco, Ashwin Gerard and Nada Lahjouji (2026). *What to Keep, What to Forget: A Rate-Distortion View of Memory Compaction in LLMs and Agents*. Frozen source SRC-STC-0035; type=paper; accessed=2026-08-05. arXiv: 2607.08032. URL: <https://arxiv.org/abs/2607.08032v1> (visited on 08/05/2026).
- Deporrois, Nicolas, Mihai A Petrovici, Walter Senn, and Jakob Jordan (2022). *Learning Cortical Representations through Perturbed and Adversarial Dreaming*. Frozen source SRC-STC-0009; type=paper; accessed=2026-08-05. DOI: 10.7554/eLife.76384. URL: <https://doi.org/10.7554/eLife.76384> (visited on 08/05/2026).
- Gerstgrasser, Matthias, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Henry Sleight, John Hughes, Tomasz Korbak, Rajashree Agrawal, Dhruv Pai, Andrey Gromov, Daniel A. Roberts, Diyi Yang, David L. Donoho, and Sanmi Koyejo (2024). *Is Model Collapse Inevitable? Breaking the Curse of Recursion by Accumulating Real and Synthetic Data*. Frozen source SRC-STC-0069; type=paper; accessed=2026-08-05. arXiv: 2404.01413. URL: <https://arxiv.org/abs/2404.01413v2> (visited on 08/05/2026).
- Golden, Ryan, Jean Erik Delanois, Pavel Sanda, and Maxim Bazhenov (2022). *Sleep Prevents Catastrophic Forgetting in Spiking Neural Networks by Forming a Joint Synaptic Weight Representation*. Frozen source SRC-STC-0011; type=paper; accessed=2026-08-05. DOI: 10.1371/journal.pcbi.1010628. URL: <https://doi.org/10.1371/journal.pcbi.1010628> (visited on 08/05/2026).
- Google Research (2026). *ReasoningBank: Enabling Agents to Learn from Experience*. Version web-capture:2026-08-05. Frozen source SRC-STC-0054; type=official release note; accessed=2026-08-05. URL:

- <https://www.research.google/blog/reasoningbank-enabling-agents-to-learn-from-experience/> (visited on 08/05/2026).
- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. Frozen source SRC-STC-0030; type=paper; accessed=2026-08-05. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685). URL: <https://arxiv.org/abs/2106.09685v2> (visited on 08/05/2026).
- Kirkpatrick, James, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell (2016). *Overcoming catastrophic forgetting in neural networks*. Frozen source SRC-STC-0002; type=paper; accessed=2026-08-05. DOI: [10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114). arXiv: [1612.00796](https://arxiv.org/abs/1612.00796). URL: <https://arxiv.org/abs/1612.00796v2> (visited on 08/05/2026).
- Letta (2026a). *Letta Code Reflection Launcher*. Frozen source SRC-STC-0060; type=official repository; accessed=2026-08-05. URL: <https://github.com/letta-ai/letta-code/blob/v0.28.18/src/cli/helpers/reflection-launcher.ts> (visited on 08/05/2026).
- (2026b). *Letta Sleep-time Multi-Agent V4*. Frozen source SRC-STC-0059; type=official repository; accessed=2026-08-05. URL: https://github.com/letta-ai/letta/blob/0.16.8/letta/groups/sleep-time_multi_agent_v4.py (visited on 08/05/2026).
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Frozen source SRC-STC-0031; type=paper; accessed=2026-08-05. arXiv: [2005.11401](https://arxiv.org/abs/2005.11401). URL: <https://arxiv.org/abs/2005.11401v4> (visited on 08/05/2026).
- Lin, Kevin, Charlie Snell, Yu Wang, Charles Packer, Sarah Wooders, Ion Stoica, and Joseph E. Gonzalez (2025). *Sleep-time Compute: Beyond Inference Scaling at Test-time*. Frozen source SRC-STC-0014; type=paper; accessed=2026-08-05. arXiv: [2504.13171](https://arxiv.org/abs/2504.13171). URL: <https://arxiv.org/abs/2504.13171v1> (visited on 08/05/2026).
- Lopez-Paz, David and Marc'Aurelio Ranzato (2017). *Gradient Episodic Memory for Continual Learning*. Frozen source SRC-STC-0004; type=paper; accessed=2026-08-05. arXiv: [1706.08840](https://arxiv.org/abs/1706.08840). URL: <https://arxiv.org/abs/1706.08840v6> (visited on 08/05/2026).
- McClelland, James L., Bruce L. McNaughton, and Randall C. O'Reilly (1995). *Why There Are Complementary Learning Systems in the Hippocampus and Neocortex*. Frozen source SRC-STC-0001; type=paper; accessed=2026-08-05. DOI: [10.1037/0033-295X.102.3.419](https://doi.org/10.1037/0033-295X.102.3.419). URL: <https://doi.org/10.1037/0033-295X.102.3.419> (visited on 08/05/2026).
- Mem0 (2026a). *Mem0 Dream*. Version web-capture:2026-08-05. Frozen source SRC-STC-0063; type=official documentation; accessed=2026-08-05. URL: <https://docs.mem0.ai/platform/features/dream> (visited on 08/05/2026).
- (2026b). *Mem0 OpenClaw Dream Gate*. Frozen source SRC-STC-0064; type=official repository; accessed=2026-08-05. URL: <https://github.com/mem0ai/mem0/blob/v2.0.13/integrations/openclaw/dream-gate.ts> (visited on 08/05/2026).
- Meng, Kevin, David Bau, Alex Andonian, and Yonatan Belinkov (2022a). *Locating and Editing Factual Associations in GPT*. Frozen source SRC-STC-0032; type=paper; accessed=2026-08-05. arXiv: [2202.05262](https://arxiv.org/abs/2202.05262). URL: <https://arxiv.org/abs/2202.05262v5> (visited on 08/05/2026).
- Meng, Kevin, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau (2022b). *Mass-Editing Memory in a Transformer*. Frozen source SRC-STC-0033; type=paper; accessed=2026-08-05. arXiv: [2210.07229](https://arxiv.org/abs/2210.07229). URL: <https://arxiv.org/abs/2210.07229v2> (visited on 08/05/2026).

- Meta AI (2026a). *HyperAgents*. Version web-capture:2026-08-05. Frozen source SRC-STC-0058; type=official release note; accessed=2026-08-05. URL: <https://ai.meta.com/research/publications/hyperagents/> (visited on 08/05/2026).
- (2026b). *Learning Personalized Agents from Human Feedback*. Version web-capture:2026-08-05. Frozen source SRC-STC-0057; type=official release note; accessed=2026-08-05. URL: <https://ai.meta.com/research/publications/learning-personalized-agents-from-human-feedback/> (visited on 08/05/2026).
- Microsoft (2026a). *Introducing STATE-Bench: A Benchmark for AI Agent Memory*. Version web-capture:2026-08-05. Frozen source SRC-STC-0056; type=official release note; accessed=2026-08-05. URL: <https://opensource.microsoft.com/blog/2026/05/19/introducing-state-bench-a-benchmark-for-ai-agent-memory/> (visited on 08/05/2026).
- (2026b). *STATE-Bench Repository*. Frozen source SRC-STC-0067; type=official repository; accessed=2026-08-05. URL: <https://github.com/microsoft/STATE-Bench/tree/4efcbf2d4fe60df04878859b692d9391f3d5b33a> (visited on 08/05/2026).
- Morris, John X., Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M. Rush, Kamalika Chaudhuri, and Saeed Mahloujifar (2025). *How much do language models memorize?* Frozen source SRC-STC-0034; type=paper; accessed=2026-08-05. arXiv: 2505.24832. URL: <https://arxiv.org/abs/2505.24832v3> (visited on 08/05/2026).
- O'Neill, Charles (2026). *Can a Language Model Learn Facts Continually in Its Weights?* Frozen source SRC-STC-0036; type=paper; accessed=2026-08-05. arXiv: 2607.11020. URL: <https://arxiv.org/abs/2607.11020v2> (visited on 08/05/2026).
- OpenAI (2025). *Memory and New Controls for ChatGPT*. Version web-capture:2026-08-05. Frozen source SRC-STC-0053; type=official release note; accessed=2026-08-05. URL: <https://openai.com/index/memory-and-new-controls-for-chatgpt/> (visited on 08/05/2026).
- (2026). *Dreaming: Better Memory for a More Helpful ChatGPT*. Version web-capture:2026-08-05. Frozen source SRC-STC-0052; type=official release note; accessed=2026-08-05. URL: <https://openai.com/index/chatgpt-memory-dreaming/> (visited on 08/05/2026).
- Ouyang, Siru, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister (2025). *ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory*. Frozen source SRC-STC-0043; type=paper; accessed=2026-08-05. arXiv: 2509.25140. URL: <https://arxiv.org/abs/2509.25140v2> (visited on 08/05/2026).
- Packer, Charles, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez (2023). *MemGPT: Towards LLMs as Operating Systems*. Frozen source SRC-STC-0013; type=paper; accessed=2026-08-05. arXiv: 2310.08560. URL: <https://arxiv.org/abs/2310.08560v2> (visited on 08/05/2026).
- Rasmussen, Preston, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef (2025). *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. Frozen source SRC-STC-0016; type=paper; accessed=2026-08-05. arXiv: 2501.13956. URL: <https://arxiv.org/abs/2501.13956v1> (visited on 08/05/2026).
- Schwarz, Jonathan, Jelena Luketina, Wojciech M. Czarnecki, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell (2018). *Progress & Compress: A scalable framework for continual learning*. Frozen source SRC-STC-0007; type=paper; accessed=2026-08-05. arXiv: 1805.06370. URL: <https://arxiv.org/abs/1805.06370v2> (visited on 08/05/2026).
- Shin, Hanul, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim (2017). *Continual Learning with Deep Generative Replay*. Frozen source SRC-STC-0003; type=paper; accessed=2026-08-05. arXiv: 1705.08690. URL: <https://arxiv.org/abs/1705.08690v3> (visited on 08/05/2026).

- Singh, Dhairyya, Kenneth A. Norman, and Anna C. Schapiro (2022). *A Model of Autonomous Interactions between Hippocampus and Neocortex Driving Sleep-Dependent Memory Consolidation*. Frozen source SRC-STC-0010; type=paper; accessed=2026-08-05. doi: [10.1073/pnas.2123432119](https://doi.org/10.1073/pnas.2123432119). URL: <https://doi.org/10.1073/pnas.2123432119> (visited on 08/05/2026).
- Tutuncuoglu, Bekir Tolga (2025). *Dream-Augmented Neural Networks: Harnessing Synthetic Sleep for Continual Learning and Zero Forgetting*. Frozen source SRC-STC-0039; type=paper; accessed=2026-08-05. doi: [10.2139/ssrn.5402490](https://doi.org/10.2139/ssrn.5402490). URL: <https://doi.org/10.2139/ssrn.5402490> (visited on 08/05/2026).
- Unknown organization (2024). *AI Models Collapse When Trained on Recursively Generated Data*. Frozen source SRC-STC-0041; type=paper; accessed=2026-08-05. doi: [10.1038/s41586-024-07566-y](https://doi.org/10.1038/s41586-024-07566-y). URL: <https://doi.org/10.1038/s41586-024-07566-y> (visited on 08/05/2026).
- (2026a). *From Player to Master: Enhancing Test-Time Learning of LLM Agents via Reinforcement Learning over Memory*. Version web-capture:2026-08-05. Frozen source SRC-STC-0051; type=paper; accessed=2026-08-05. URL: <https://openreview.net/forum?id=gNWNtstp3r> (visited on 08/05/2026).
- (2026b). *Memory-Statistics Tradeoff in Continual Learning with Structural Regularization*. Version web-capture:2026-08-05. Frozen source SRC-STC-0050; type=paper; accessed=2026-08-05. URL: <https://openreview.net/forum?id=qfEqXJnlB4> (visited on 08/05/2026).
- Zep (2026a). *Graphiti: Temporal Knowledge Graphs for Agent Memory*. Frozen source SRC-STC-0065; type=official repository; accessed=2026-08-05. URL: <https://github.com/getzep/graphiti/blob/aab852df94413fd0d55cbea2b7886173020281d5/README.md> (visited on 08/05/2026).
- (2026b). *Zep Episodes*. Version web-capture:2026-08-05. Frozen source SRC-STC-0066; type=official documentation; accessed=2026-08-05. URL: <https://help.getzep.com/episodes> (visited on 08/05/2026).
- Zweiger, Adam, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal (2025). *Self-Adapting Language Models*. Frozen source SRC-STC-0029; type=paper; accessed=2026-08-05. arXiv: [2506.10943](https://arxiv.org/abs/2506.10943). URL: <https://arxiv.org/abs/2506.10943v2> (visited on 08/05/2026).